

开源软件组件漏洞检测与自动修复技术研究综述

张旭明, 史涯晴, 黄松, 王兴亚, 胡津昌, 陆江涛

引用本文

张旭明, 史涯晴, 黄松, 王兴亚, 胡津昌, 陆江涛. [开源软件组件漏洞检测与自动修复技术研究综述](#)[J]. 计算机科学, 2025, 52(6): 1-20.

ZHANG Xuming, SHI Yaqing, HUANG Song, WANG Xingya, HU Jinchang, LU Jiangtao. [Survey of Open-source Software Component Vulnerability Detection and Automatic Repair Technology](#) [J]. Computer Science, 2025, 52(6): 1-20.

相似文章推荐 (请使用火狐或 IE 浏览器查看文章)

Similar articles recommended (Please use Firefox or IE to view the article)

[业务流程模型相似度研究综述](#)

Review on Similarity of Business Process Models

计算机科学, 2023, 50(6): 338-350. <https://doi.org/10.11896/jsjcx.220700061>

[基于交叉指纹分析的公共组件库特征提取方法](#)

Feature Extraction Method for Public Component Libraries Based on Cross-fingerprint Analysis

计算机科学, 2023, 50(1): 373-379. <https://doi.org/10.11896/jsjcx.211100121>

[以太坊智能合约模糊测试技术研究综述](#)

Survey of Ethereum Smart Contract Fuzzing Technology Research

计算机科学, 2022, 49(8): 294-305. <https://doi.org/10.11896/jsjcx.220500069>

[基于区块链与改进CP-ABE的众测知识产权保护技术研究](#)

Study on Crowdsourced Testing Intellectual Property Protection Technology Based on Blockchain and Improved CP-ABE

计算机科学, 2022, 49(5): 325-332. <https://doi.org/10.11896/jsjcx.210900075>

[基于多目标优化的测试用例优先级排序方法](#)

Test Case Prioritization Based on Multi-objective Optimization

计算机科学, 2020, 47(6): 38-43. <https://doi.org/10.11896/jsjcx.191100113>

开源软件组件漏洞检测与自动修复技术研究综述

张旭明¹ 史涯晴¹ 黄松¹ 王兴亚^{1,2} 胡津昌¹ 陆江涛¹

¹ 陆军工程大学指挥控制工程学院 南京 210007

² 南京工业大学计算机与信息工程学院 南京 211816

(1426389266@qq.com)

摘要 不可阻挡的软件组件化趋势与多人协同作业模式的开发过程促使开源软件供应链逐渐形成,在蓬勃发展的开源软件带来巨大便利的同时,开源漏洞随着供应链悄然而至,威胁着软件系统的安全性和可靠性。软件成分分析可对维护脆弱的开源软件供应链的安全起到实质性的支撑作用,它通过开源软件组件漏洞检测与自动修复两个核心功能发现并修复软件项目中潜在的漏洞和风险。为加深相关研究人员对软件成分分析在安全漏洞方面的实践的了解,文中梳理归纳了近年的开源软件组件漏洞检测与自动修复技术的研究进展与成果;进一步地,从用户视角出发,基于8个分析维度对目前常见的8种软件成分分析工具进行了总结与探析;最后,探讨了开源软件组件漏洞检测与自动修复技术现存的挑战并展望了未来可能的发展方向。

关键词: 开源软件供应链;软件成分分析;组件漏洞检测;自动修复

中图分类号 TP311

Survey of Open-source Software Component Vulnerability Detection and Automatic Repair Technology

ZHANG Xuming¹, SHI Yaqing¹, HUANG Song¹, WANG Xingya^{1,2}, HU Jinchang¹ and LU Jiangtao¹

¹ College of Command and Control Engineering, Army Engineering University of PLA, Nanjing 210007, China

² College of Computer Science and Technology, Nanjing Tech University, Nanjing 211816, China

Abstract The unstoppable trend of software componentization and the development process of collaborative work mode by multiple individuals have led to the gradual formation of open-source software supply chains. While flourishing open-source software brings great convenience, open-source vulnerabilities quietly emerge along the supply chain, posing a threat to the security and reliability of software systems. Software composition analysis technology can provide substantial support for maintaining the security of fragile open-source software supply chains. It discovers and fixes potential vulnerabilities and risks in software projects through two core functions: open-source software component vulnerability detection and automatic repair. To deepen the understanding of relevant researchers on the practical application of software component analysis in security vulnerabilities, this paper reviews and summarizes the research progress and achievements in recent years on open-source software component vulnerability detection and automatic repair technologies. Furthermore, starting from the users' perspective, a summary and analysis of eight common software composition analysis tools based on eight analysis dimensions are provided. Finally, the existing challenges of open-source software component vulnerability detection and automatic repair technology are discussed, and their possible future development directions are explored.

Keywords Open-source software supply chain, Software component analysis, Component vulnerability detection, Automatic repair

1 引言

由于软件应用中功能复杂性的快速增长,软件组件化已经成为软件开发中不可抗拒的趋势,这促使了开源软件的蓬勃发展。随着开源代码在代码库中的比重不断增加,几乎所有企业都使用免费的开源代码来开发自己的软件产品。开源代码能够帮助软件开发企业和组织极大地缩短软件开发的周期。由此可见,开源代码已经成为软件开发过程中的重要

组成部分,也成为当今网络空间的基石。根据综合性网络安全公司奇安信在2023年发布的《2023中国软件供应链安全分析报告》^[1],2021年底和2022年底,主流开源软件包生态系统中开源项目总量分别为4395386个和5499977个,一年间增长了25.1%;截至2022年底,主流开源软件包生态系统中平均每个开源项目有12.6个版本,较前两年报告的11.8个和10.2个呈持续增长趋势;2020年至2023年的漏洞密度和高危漏洞密度均呈现出逐年上升的趋势。由此不难看出,

开源代码和开源社区在过去的几年中迎来了大规模的增长, 开源漏洞也随之爆发式增长, 开源软件的潜在危险也激增。

而今, 信息和通信技术得到了迅猛发展, 人们能够轻松地获取和共享大量的信息。信息技术已经深入到人们日常生活的方方面面, 个人以及各类组织机构对软件有着强烈的依赖性, 对社会、经济和文化产生了深远的影响。每个软件都有自己的供应链, Ji 等^[2]定义了开源软件供应链的 4 个关键环节, 包括组件开发环节、应用软件开发环节、应用软件开发分发环节和应用软件使用环节, 并总结了各个环节的威胁渗入点。开源软件供应链作为一个具有巨大吸引力的突破点, 对于那些对非法侵犯、利用、窃取甚至破坏政治、经济利益感兴趣的人或者组织而言特别具有吸引力。在一个软件生态系统中被广泛使用的第三方库, 其出现的漏洞可能导致整个系统存在风险, 进而造成巨大的经济损失和严重的信息泄露。正如 Log4shell^[3]事件, 其中 Apache Log4j 是一个被 Apache Tomcat 和 Apache Spark 等众多应用程序使用的 Java 日志记录库, 然而有安全研究人员在 2021 年 12 月发现该软件存在远程代码执行漏洞, 这可能影响到难以计数的使用了 Log4j 的应用程序。

开源软件(Open-Source Software, OSS)组件在软件开发过程中展现了显著的优势以及“开源”一词的推广, 加之开源软件供应链的脆弱性, 这两重因素使得相关人员在享受使用开源软件的便利时不得不担忧开源漏洞的危害并不断为修复开源漏洞做大量工作。尽管开源软件漏洞给开发者和用户带来了巨大困扰与挑战, 但是对于开源安全问题他们也没有足够重视。开发人员安全平台 Snyk 和 Linux 基金会的研究人员在他们的《2022 开源安全状况》^[4]报告中透露, 应用程序中未解决的超危漏洞的平均数量为 5.1; 在未对“OSS 的安全策略是如何声明的”做任何要求的情况下, 只有 49% 的组织拥有涵盖 OSS 开发或使用的安全策略; 只有 59% 的组织认为他们的开放源代码软件是安全的或高度安全的。Wermke 等^[5]在与开发人员、架构师、工程师的访谈中, 发现受访者大量使用开源组件、开源社区, 并且只有大约一半的人提到了他们的开发过程中设立了特定于安全的角色。

为了应对开源漏洞的威胁并促进开源漏洞治理, 开源软件供应链的各个环节涉及的研究人员为之付出了巨大努力。其中, 由于软件成分分析(Software Component Analysis, SCA)能暴露软件项目中的第三方组件, 并能识别由软件项目中的第三方组件而引入软件项目并影响软件项目本身的已公开漏洞, 现在 SCA 工具已被开发人员以及用户等利益相关者广泛应用。SCA 工具从开源软件供应链的各个环节为开源软件供应链的安全提供了保障。对于组件开发环节和应用软件开发环节的开发人员来说, 他们为提高开发效率通常会使用大量 OSS 组件, 因而格外关注使用的 OSS 组件的安全性; 对于应用软件开发分发环节和应用软件使用环节的相关人员, 他们需要了解分发或使用的应用软件中使用的 OSS 组件是否安全, SCA 工具很大程度上缓解了他们的顾虑。SCA 工具除了确保软件安全性方面有着至关重要的作用, 在非安全方面也有着巨大的应用价值, 如优化开源软件的开发流程。

通过 SCA, 可以识别和跟踪软件中使用的开源组件及其相关信息, 帮助开发团队更好地管理和维护软件的成分, 这可以提高开发效率、降低成本, 同时确保软件的质量和可靠性。

为加深相关研究人员对 SCA 的了解, 推进 SCA 工具的研究与发展, 同时考虑到现有工作缺乏对 SCA 工具中的 OSS 组件漏洞检测与自动修复的方法进行系统性的分类与讨论, 本文从 OSS 组件漏洞检测与自动修复两方面总结相关文献的理论、实践、适用场景、贡献以及局限性并进行全面的洞察, 希望对 SCA 工具的下一步发展起到指导作用。

1.1 研究问题

SCA 是信息技术和软件工程领域的实践, 它用于分析目标项目中引入的开源组件, 并检测这些组件是否包含安全漏洞^[6-7]、是否是最新的, 以及是否具有许可要求^[8]。在文献[6]中, SCA 的工作流程分为以下 4 个步骤:

1) 扫描(也称依赖解析): 扫描目标项目以识别使用的 OSS 组件。

2) 比对: 将检测到的 OSS 组件与漏洞数据库进行比对, 以寻找目标项目中潜在的安全漏洞。

3) 分析: 评估检测到的易受攻击组件(特别是安全漏洞)的风险, 并提供修复指导。

4) 报告: 将分析结果以规范格式呈现给用户。

为总结可用于支撑 SCA 进一步发展的技术的研究进展, 本文拟从 SCA 工作流程着手, 调研 OSS 组件漏洞检测与自动修复技术的研究进展。本文所讨论的 OSS 组件漏洞检测可以支撑上述扫描和比对环节, 这是因为在对 OSS 组件漏洞检测的相关文献进行总结时, 发现大多数文献将扫描和比对放在一起进行研究。本文讨论的自动修复工作可以支撑上述分析环节。而上述报告环节不在本文的讨论范围内。

1) OSS 组件漏洞检测。传统的漏洞检测主要利用静态分析或动态测试来分析、挖掘或检测源代码中潜在的 0day 安全漏洞。本文聚焦于开源供应链场景下, 由 OSS 组件带来的 1day 漏洞。这项工作旨在识别目标软件中所使用的 OSS 组件, 以及在哪些 OSS 组件中受公开漏洞影响的 OSS 组件。目标软件的 OSS 组件依赖关系在用户和开发人员之间缺乏明确性, 用户无法准确了解目标软件的全部依赖关系, 而开发人员仅能获悉其第一层依赖关系。在依赖关系不透明的环境下, 用户和开发人员难以确切了解他们所用或开发的目标软件的安全状况。因此, 进行 OSS 组件漏洞检测尤为必要。

在进行 OSS 组件漏洞检测的过程中, 通常首先利用包管理器(如 npm, maven, pip 等)以及依赖解析技术来获取目标软件直接依赖的 OSS 组件。随后, 将所获得的直接依赖的 OSS 组件视为目标依赖项, 重复进行直接依赖解析过程, 直至不再存在直接依赖关系, 从而形成目标软件的依赖树, 最终完成对目标软件的 OSS 组件分析过程。最后, 利用漏洞数据库中的漏洞信息(包括漏洞描述、受影响软件、影响范围等)对目标软件的 OSS 组件进行漏洞检测, 以获取受漏洞影响的 OSS 组件列表。

2) 自动化修复。该项工作旨在针对上一环节中检测出的

目标软件中的 OSS 组件(不一定是存在漏洞的 OSS 组件,还可能是不兼容或者过时的 OSS 组件),通过自动化方式快速、有效地将这些 OSS 组件修复至安全且兼容的版本。在软件生命周期中,自动化修复具有至关重要的地位,它可以提升软件的安全性和可靠性,降低潜在安全风险。

在进行自动化修复的过程中,针对受漏洞影响的 OSS 组件,可以利用漏洞数据库获取已公开的补丁信息,以确定该组件的安全版本范围。随后,通过兼容性检查,排除与目标软件依赖树中其他 OSS 组件不兼容的版本,从而确定既安全又兼容的版本。最终,将受漏洞影响的 OSS 组件修复至安全且兼容的版本。

上述工作流程如图 1 所示,需要注意的是,上述 SCA 工作流程是一般情况下的典型流程,旨在为读者初步介绍 SCA

的工作原理。然而,这一流程并不涵盖所有情况。例如,当目标软件以二进制文件形式存在时,无法直接进行依赖解析。针对这类目标软件,通常需要借助反汇编技术或动态运行目标软件来检测其是否受到漏洞影响。因此,在 OSS 组件漏洞检测及自动修复技术方面的当前研究进展将在第 3 章和第 4 章中进行介绍。

通过 OSS 组件漏洞检测与自动化修复两个环节,SCA 工具可以及时发现和修复软件中的安全问题,从而很大程度上确保软件的安全性和可靠性。截至目前,大部分 SCA 工具的功能已经相对完善,除以上核心功能外,它还提供软件许可证合规性分析等功能。本文专注于总结 OSS 组件漏洞检测和自动化修复两方面的研究进展,特别关注它们在安全领域的最新研究成果和应用前沿。

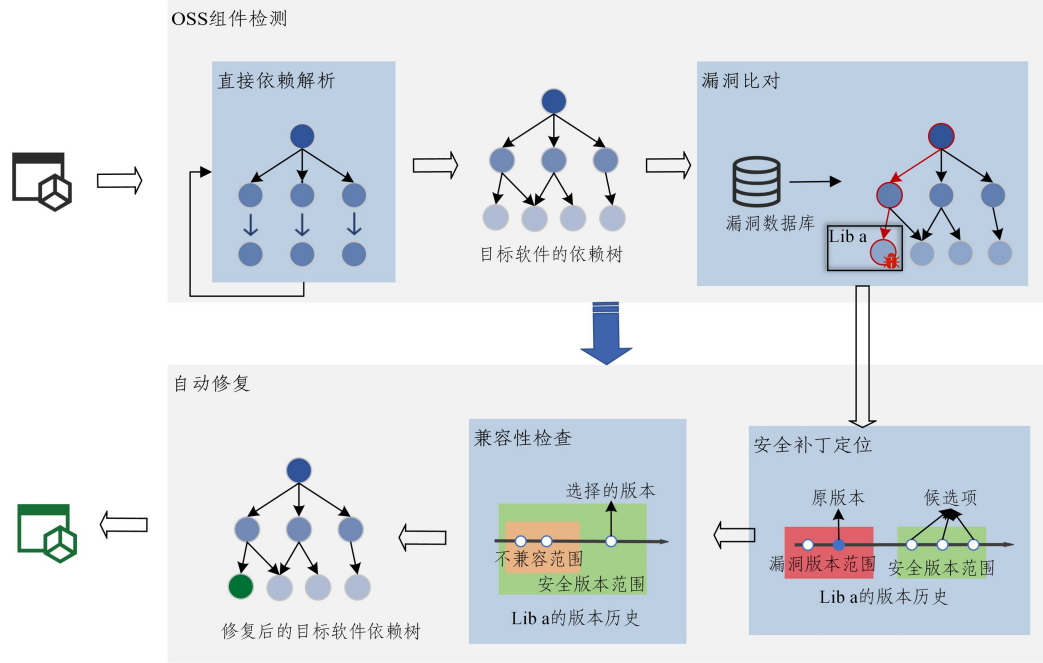


图 1 SCA 工具的核心工作流程图

Fig. 1 Core workflow of SCA tools

根据上述内容,本文旨在回答以下问题:

RQ1:现有的 OSS 组件漏洞检测方法有哪些?

大多数研究借助依赖元数据分析开源软件的依赖关系并在此基础上检测漏洞。此外,也有研究借助源代码文件以及二进制文件进行分析并检测漏洞。同时,研究者们也通过利用这些方法得到的漏洞检测结果发现了许多有趣的事实。这些方法及发现将在第 3 章进行讨论。

RQ2:现有的自动化修复方法有哪些?

在检测出漏洞后,自动化修复也是 SCA 技术的重要一环。借助一些官方公告、漏洞数据库、安全报告等信息源,通常修复建议或补丁信息可以被获取到。然而,对于多源、不一致、缺失的修复建议或补丁信息,如何选择合适的修复建议进行自动化修复?此外,不少 SCA 工具在自动化修复时忽略了兼容性,导致更新失败或更新不可用,这种情况如何避免?这些问题将会在第 4 章一一阐述。

RQ3:常见的 SCA 工具有哪些?

探索现有的 SCA 工具,分析它们的特点和用途,这有

助于进行新的研究。这部分内容将在第 5 章中详细讨论。

1.2 研究方法

本文遵循系统分析法,综合调研其背景,包括开源漏洞等相关知识及相关研究,并对 OSS 组件漏洞检测与自动化修复技术进行了详细的介绍与归纳。基于本文的目的,本文依托完善的 SLR 指南^[9],制定了以下文献检索策略,以确定可用于回答 1.1 节的 3 个问题,并减少可能影响本文结果的其他因素。

本文在出版书籍、公开的会议论文以及期刊上,主要对 2018—2023 年间现有的关于开源软件漏洞分析技术的文献、综述进行收集并分析,以确定各研究的差距。按照以上原则,具体分为以下步骤。

1)根据中国计算机学会推荐的国际学术会议和期刊目录,本文以软件安全领域的会议与期刊(包括 TOSEM, TSE, FSE/ESEC, ASE, ICSE, ISSTA),以及网络与信息安全领域的会议与期刊(包括 CCS, S&P, USENIX Security)为关键检索对象。为确保检索的结果与本文研究内容贴合密切,本文

约定 7 个关键检索词,包括 open-source software, vulnerability, dependency, fix, patch, compatibility, Software Component Analysis。

2)为避免遗漏未发表在步骤 1 中 9 项会议或期刊中的优秀文献,本文在 ACM 文献数据库、IEEE Xplore 文献数据库、Science Direct 文献数据库、SpringerLink 电子期刊数据库 4 个先进的文献存储库中使用步骤 1 约定的 7 个关键检索词进行搜索,并在标题、摘要、引用中搜索。

3)为防止前两个步骤检索相关文献时的疏漏,本文遵循跟踪检索法,对前两个步骤选取出来的文献提取参考文献,在它们的参考文献中选取与本文研究相关的文献作为本文的参考文献。

4)最后手工浏览前 3 个步骤选取的文献,检查标题和摘要,收集并整合相关研究,以确保它们与本文的重点一致。

值得注意的是,通过步骤 3 筛选出来的部分文献的发表年份不在前面所约定的时间范围,通过人工阅览,本文保留了 1 篇 2014 年、1 篇 2015 年、1 篇 2016 年以及 4 篇 2017 年的文献作为参考文献。经过上述 4 个步骤,最终 71 篇文献被纳入本文的参考文献。在这 71 篇文献中,有 43 篇文献提出了可支持 OSS 组件漏洞检测或自动修复技术的新方法,有 28 篇文献是关于 SCA 的背景动机或对开源安全的一些实证研究。上述文献发表的会议或期刊分布情况以及发表年份分布情况分别如图 2 和图 3 所示。

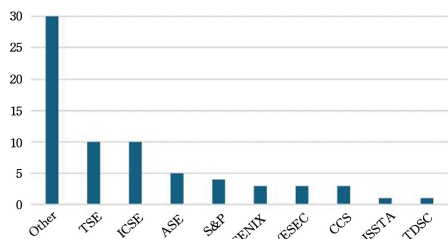


图 2 选取文献的发表会议或期刊分布情况

Fig. 2 Distribution of publication conferences or journals for selected literature

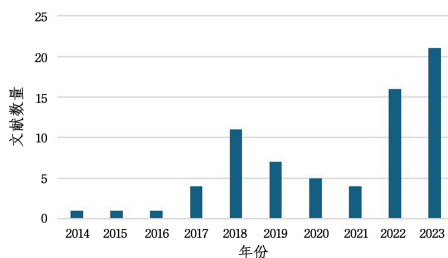


图 3 选取文献的发表年份分布情况

Fig. 3 Distribution of publication years for selected literature

文献选取对本文有效性的威胁包括以下 3 点:1)在文献选取阶段,由于在文献数据库上执行的基于关键检索词的查询,可能会错过一些优秀的论文,导致它们在本文中未涉及;2)文献数据提取与分析工作量较大,不可避免地会遗漏一些错误;3)本文主要针对 2018—2023 年收集的研究论文,在此期间,由于安全事件频发,安全问题愈发

被重视,开源软件漏洞管理技术在这一时期显著加强,本文结果不能保证涵盖所有的相关研究。

1.3 文章结构

本文第 2 章讨论了研究背景及相关工作;第 3 章总结了与 OSS 组件漏洞检测方法相关的研究,并对这些方法科学地进行分类;第 4 章介绍并归纳了与自动化修复相关的研究,包括安全补丁定位方法与兼容性检查方法两部分;第 5 章以用户角度探讨了现有的 SCA 工具;第 6 章探讨了 OSS 组件漏洞检测与自动化修复技术现有难点并对未来工作进行了展望;最后对全文进行了总结。

2 研究背景及相关工作

2.1 开源漏洞

开源漏洞指存在于开源生态系统中的开源软件的可能被攻击者利用从而导致安全事件的漏洞或弱点。根据 OWASP Top Ten^[10],使用易受攻击或过时的组件已成为严重威胁 Web 应用程序安全的风险,并从第九名上升到第六名。在其他研究中,OSS 组件也称作第三方库、依赖项或第三方组件(本文中将这 4 个词视为相同意思),它们都指在开发过程中引入到项目的已经被公开源代码的外部代码或功能模块,通常被放在代码托管平台或各自编程语言对应的包存储库中。在开发过程中,添加开源软件组件可以避免重复的工作^[11],但 OSS 组件的漏洞也会被引入到开发的项目中。将最先引入漏洞的软件视作源漏洞软件,则使用源漏洞软件作为 OSS 组件的软件也会成为漏洞软件,这些漏洞软件又会作为 OSS 组件继续影响别的软件并将其转变为漏洞软件。如此,会在整个开源生态系统中形成错综复杂的开源依赖关系网络^[12]。一旦源漏洞软件成为攻击者的目标,与此源漏洞软件存在依赖关系的软件均会受到影响。

漏洞的生命周期通常可以分为以下几个阶段。

1)发现阶段:个人或组织发现并确认漏洞。

2)上报阶段:发现者将漏洞报告给相关方,如软件开发者、供应商或安全组织等。

3)修复阶段:接收到漏洞报告后,相关的组织或开发团队开始进行漏洞修复工作。

4)发布补丁阶段:修复漏洞后,软件开发者或供应商会发布相应的安全补丁或更新版本。

5)通知用户阶段:当补丁发布时,漏洞软件的厂商会第一时间通过安全公告、电子邮箱、官网新闻等方式通知用户和受影响的组织,提醒他们及时更新软件并修复漏洞。

在漏洞的整个生命周期中,攻击者可以利用的时间点很多。文献[13]研究了漏洞引入、发现和修复的持续时间,文献[14]调查了漏洞修复的发布、采用和传播趋势,文献[15]研究了安全修复与安全发布、安全发布与咨询发布之间的时间差,文献[16]调研了漏洞版本与其修复版本之间的时间滞后。它们的研究工作均证实了下游项目漏洞修复存在延迟,不仅修复前的漏洞可以被攻击者介入,而且修复到发布阶段之间有延迟,并且补丁发布并不意味着“痊愈”(用户可能不会选择补丁进行修复),这使得攻击者有更多的机会可以介入。

2.1.1 漏洞的变化趋势

了解漏洞变化趋势的作用为:1)有助于开发人员更好地修补已知漏洞并预防新漏洞的出现;2)有助于相关组织及个人更好地了解漏洞当前的威胁情况,提高安全意识并加强安全防护意识;3)有助于安全团队及时地更新安全策略和措施,以更好地保护系统和数据。

为研究漏洞的变化趋势,本文主张将开源漏洞和闭源漏洞综合考虑。这是因为开源软件可能会依赖于闭源的第三方组件,导致闭源软件中的漏洞传播到开源软件中。国家漏洞数据库(NVD)是安全研究人员最常用的漏洞数据库之一^[17-18],该漏洞数据库收录的漏洞相对全面,并且漏洞数据结构化良好,因此本文选择利用NVD来研究漏洞数量的变化趋势。

具体地,在2024年1月,本文对NVD进行了快照,收集了来自NVD的漏洞信息,并根据漏洞记录的创建时间进行了统计,最终统计结果如图4所示。为了更好地反映近几年漏洞的变化趋势,本文重点关注2018年至2023年这6年的漏洞数量逐年变化情况,这也与本文的参考文献的时间选取范围(2018年至2023年)保持一致,因此1999年至2017年的数据并未详细给出。注意,CVE计划是1999年开始的,NVD的数据来源是CVE,故而漏洞记录的创建时间是1999年开始的。

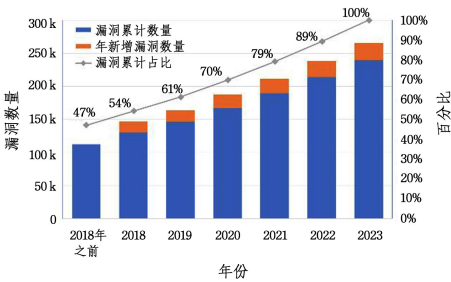


图4 漏洞数量随时间变化趋势图

Fig. 4 Trend chart of vulnerability quantity over time

从图中显示的结果来看,不难发现:

1)从2019年开始,每一年的新增漏洞数量基本都多于前一年。特别是2022年与2023年这两年,年新增漏洞数量呈明显的上涨趋势,一个可能的原因是:2021年爆发的Log4shell事件使得开源安全领域受到更多个人和组织的关注与重视,自此事件爆发后,更多的漏洞得以被更多个人或组织挖掘以及公开。

2)2018年及2018年之前的漏洞累计数量占2023年漏洞累计数量的一半左右,换言之,2019年到2023年这5年间新增的漏洞数量大约和1999年到2018年这20年间新增的漏洞数量相同,可以认为近五年的漏洞数量激增。

2.1.2 开源漏洞的传播与影响

先前的工作^[19-23]证实了漏洞广泛存在,漏洞修复时间长且漏洞软件的下游项目修复时间更长,漏洞的传播路径不止一条,漏洞软件的修复重视程度不足,单个漏洞可以影响大型代码库等,这些证据足以说明开源软件的广泛使用、漏洞的传播性导致开源生态系统受漏洞影响范围越来越广。这些工作

的详细内容如下。

Li等^[19]在研究补丁的生命周期中发现三分之一的漏洞从引入到最终被修复的时间超过了3年。并且补丁也并不总是可靠的,近5%的安全修复对相关软件产生了负面影响,7%的开发人员未能完全修复相关漏洞。

Düsing等^[20]在NuGet.org、NPM Registry以及Maven Central存储库中深入研究开发人员处理漏洞补丁的方式以及更新依赖项的情况,发现尽管补丁通常在漏洞发布前已经发布,但是超四分之一的漏洞并未打补丁,其中不乏高危漏洞。根据存储库的不同,应用程序受传递依赖影响的程度也显著不同,其中Maven Central存储库可能受深度高达25的传递依赖影响。结果也表明即使部分库受直接依赖影响但其中大部分库都不会升级任何一个依赖项,并且开发人员可能依赖于依赖项自动更新。

Zerouali等^[21]针对npm和RubyGems包的定量漏洞,分析了漏洞在依赖网络中的影响。他们是第一个研究RubyGems包漏洞的普遍性、披露及修复时间和第一个报告漏洞披露持续时间如何随时间演变的团队。他们发现,在其研究范围内,npm中的漏洞呈指数级增长,RubyGems中的漏洞呈线性级增长,披露时间长达2年且修复时间长达4年的漏洞超过50%。

Iannone等^[22]分析开发人员如何引入以及删除漏洞。通过实验结果发现,大部分应用程序引入了不止一个漏洞,大多数项目在项目新建、修复错误或重构时便会引入漏洞,工作量较大的开发人员也更易在工作中引入漏洞,并且漏洞从引入到修复时间跨度极大,漏洞在项目中的生存率极高。

Wang等^[23]观察到大量软件包没有重视自己在传播链中的影响力,导致下游项目修复滞后,通过对npm生态系统深入调查发现有45148个阻塞包和358422个阻塞链致使983336条依赖路径的修复滞后,其中不乏与高危漏洞相关的阻塞链。

以上研究探索了开源漏洞的引入、传播以及修复的途径、规律与影响,钻研了不同包生态系统下开源漏洞的特异性,这些研究的发现与结论提供了对漏洞检测、漏洞修复等方面极具价值的信息与启示,可以帮助安全研究人员更好地理解漏洞是如何被利用以及如何被修复的,也可用于指导SCA工具的设计。

2.1.3 开源漏洞的传递性与可达性

传递性与可达性是开源漏洞的两个重要性质。分析漏洞的传递性有助于检测隐藏在能直观看到的依赖项之外的漏洞,使软件项目具有更强的抗攻击能力。而分析漏洞的可达性,有助于了解漏洞是否在实际运行过程中被执行,减少误报情况发生,增加OSS组件漏洞检测结果的可信程度。

识别软件项目的直接依赖与间接依赖有助于进一步分析漏洞的传递性。直接依赖指一个软件组件明确地依赖于另一个组件。例如,在一个Java项目中,如果开发人员将一个OSS组件添加到项目的依赖配置中,那么这个库就是该项目的直接依赖。直接依赖是开发人员明确使用并直接引入到其代码中的组件。间接依赖指通过直接依赖而间接引入的依赖

关系。当开发人员使用的直接依赖项本身又依赖于其他组件时,这些被直接依赖项所依赖的组件就是间接依赖项。换言之,间接依赖是由使用的直接依赖所传递或引入的其他组件。图 5 给出了软件 A 的依赖树。组件 B、C、D 与软件 A 是直接依赖关系,组件 E、F 与软件 A 是间接依赖关系。

可达性,也称作可访问性,它指一个对象或模块在程序实际执行过程中能否通过一条依赖路径调用或访问到另一个对象或模块。如果一个对象或模块无法通过现有的依赖关系路径调用或访问到另一个对象或模块,那么就说这个依赖关系是不可达的。图 6 是软件 G 的 OSS 组件可达性示例图,OSS 组件 H、M、N 被引入软件 G 的开发过程,组件 H 和 N 在程序执行过程中被使用,这表明在程序实际执行过程中,软件 G 能分别通过 $G \rightarrow H$ 和 $G \rightarrow N$ 两条路径到达组件 H 和 N。而组件 M 在程序执行过程中并未被使用,这表明在程序实际执行过程中,软件 G 不能通过 $G \rightarrow M$ 这条路径到达组件 M,故认为组件 H 和 N 可达,且组件 M 不可达。

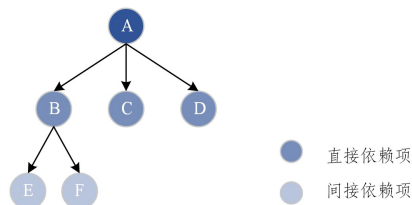


图 5 软件 A 的依赖树示例

Fig. 5 Example of dependency tree for software A

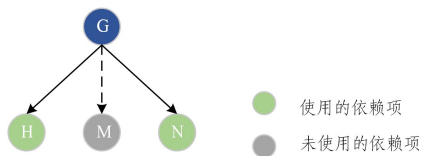


图 6 软件 G 的 OSS 组件可达性示例

Fig. 6 Example of OSS component accessibility for software G

2.2 依赖元数据

依赖元数据指与软件的依赖项相关的信息,包括但不限于以下内容。

1)安全公告:厂商或第三方组织发布的以自然文本描述的漏洞相关信息,通常包括漏洞名称、影响软件及范围等。对于大型软件项目来说,厂商通常有自己的官网,厂商安全公告可以在官网查阅,如 Apache^[24],Microsoft^[25]等。相较于专注提供自家软件产品安全信息的厂商,第三方组织提供的安全公告包含的信息更为广泛,涵盖多个软件产品,如 openwall^[26]等。

2)漏洞数据库:收集并整合来自厂商安全公告、第三方安全公告平台、漏洞跟踪平台等多源的漏洞信息,并分析和归档这些信息,为用户提供受漏洞影响的软件名称、受影响的版本范围、详细描述、严重程度评分、修复建议等漏洞相关信息。漏洞数据库的详细介绍见 2.3 节。

3)代码托管平台数据:代码托管平台是存储和管理软件项目的源代码、相关资源和文件的平台。在开发过程中,代码托管平台可以辅助开发人员进行协作编辑、维护和跟踪代码

的更改,极大优化多人协同作业的过程,因此绝大多数开源项目的开发团队会选择代码托管平台作为存储库。常见的代码托管平台有 Github, GitLab, Gitee 等。通过代码托管平台,可以获取到软件项目源代码、commit 历史、版本列表、issue 历史、PR 请求列表等相关信息,而且代码托管平台还提供给开发人员与其他开发人员等利益相关者一个讨论与交流的平台。其中,commit 历史记录记录了每个 commit 的提交者、日期、代码更改、涉及文件等信息,issue 历史记录记录了每个 issue 的名称、描述、修复链接等信息,PR 请求列表记录了每个 PR 请求的标题、描述、状态等信息。issue 和 PR 请求是项目利益相关者报告和讨论软件问题和代码更改的主要方式^[27],查阅这些信息是一个了解项目漏洞和补丁相关信息的重要途径。

在 OSS 漏洞检测以及修复阶段,都能看见依赖元数据的身影。例如利用代码托管平台上应用程序的配置文件获取到应用程序的依赖项以及依赖项版本,将依赖项以及依赖项版本与漏洞数据库中受漏洞影响的软件名称及影响版本进行比较,可以检测依赖项是否存在漏洞。如果某个依赖项的版本与已知漏洞相关联,那么它可能是一个存在漏洞的依赖项。除此之外,还可以在依赖项官网或第三方平台查阅依赖项的安全公告来了解它的安全性。漏洞数据库与安全报告对每个漏洞几乎都提供了修复建议或补丁链接,依赖项存在漏洞的应用程序可以通过这些修复建议或补丁链接修复依赖项。依赖元数据对研究 OSS 漏洞领域的重要性与贡献不止于此,本文将在第 3 章和第 4 章中结合大量的文献阐述实际研究中依赖元数据是如何被使用的。

2.3 漏洞数据库

在软件的生命周期中,漏洞数据库提供已公开的漏洞信息,给漏洞扫描、静态代码分析、漏洞修复等许多涉及到漏洞管理的工具或流程提供数据支撑。研究人员通常将漏洞数据库视为可信的信息知识资源,而不将它们与其他软件生命周期工作完全集成。漏洞数据库的建立,从行业角度来看,有助于应对日益增长的软件漏洞和攻击;从开发人员、安全人员等相关利益涉及者角度来看,有助于他们更好地查阅、了解及利用漏洞的相关信息。

文献^[18]选取了 2001—2018 年软件工程领域的 99 篇文献,研究了这 17 年之间的常用漏洞数据库以及使用漏洞数据库的安全主题。Li 等^[17]选取了 2022 年之前的 69 篇文献,从流行漏洞数据库、使用数据库的目标、采用的其他信息源等方面,系统且深入研究了可用数据库的情况。这两项研究的结论都指出,近年来采用的流行数据库(按照排名)有 NVD, CVE 等,并且大多数研究利用这些数据库进行分析,如漏洞影响范围分析、漏洞演变趋势分析等。

常见漏洞和披露(Common Vulnerabilities and Exposures, CVE)计划^[28]的主要目标是识别、定义和编录发现的安全漏洞,它赋予了每个漏洞的唯一标识符,并为每个漏洞提供一份 CVE 记录,包含漏洞描述、影响软件、影响软件版本等信息。当个人或组织发现新漏洞并向 CVE 计划参与者报告漏洞,CVE 计划参与者会请求一个全新的 CVE 标识符,表示该

漏洞的初始状态(此时还未披露),在 CVE 计划参与者提交漏洞详细信息后,负责分配 CVE 编号和管理 CVE 条目的组织会将此 CVE 发布并加入 CVE 列表中。

国家漏洞数据库(National Vulnerability Database, NVD)^[29]是美国国家标准与技术研究院(NIST)基于 CVE 构建的一个集成了所有公开可用的美国政府漏洞资源并提供行业资源参考的全面的网络安全漏洞数据库,并且该数据库是与 CVE 列表同步的漏洞数据库。除了常规的漏洞信息,NVD 还包含了 CPE(用于标识和识别计算机系统、软件和硬件的标准化命名方案,通常由厂商名称、产品名称和版本号组成)、CWE(关于软件和硬件中常见弱点列表,描述了漏洞的根本原因)、CVSS(用于评估和量化计算机系统和软件中漏洞严重性,由基础得分、影响度和可利用性 3 个度量标准组成)的重要信息,以提供更全面的漏洞管理和风险评估支持。

除了 NVD 与 CVE 外,还有其他知名的漏洞数据库也被广泛用于开源软件安全领域的各项研究,如 CNVD^[30],CNNVD^[31],OSVDB^[32],GHSA^[33]等。

2.4 版本控制系统与版本控制规则

版本控制系统是一种用于跟踪和管理源代码版本的工具。常见的版本控制系统包括 Git,SVN 等。当对代码进行增加、删除、修改等操作或版本更新等功能时,可以用版本控制系统记录文件、函数、方法、代码的历史变更;并且版本控制系统提供了分支管理、合并和版本回滚等功能。版本控制系统通常与代码托管平台结合使用,如 Git 与 GitHub、Git 与 Gitee、SVN 与 Apache Subversion,这极大地发挥出了版本控制系统与代码托管平台的便于协作、高效管理的优势,呈现一加一大于二的效果。

版本控制规则的出现是为了突破依赖地狱的困境。在软件管理中,复杂的依赖关系导致版本依赖被锁死或版本混乱,意味着项目正处于依赖地狱。版本控制规则用一组简单的规则及条件来约束版本号的配置和增长。这些规则是根据(但不局限于)已经被各种闭源、开源软件所广泛使用的惯例所设计的,它为软件版本号提供了一致性和可读性,通常与版本控制系统一起使用,以帮助开发者管理软件版本和向用户传达变更信息。常用的版本控制规则有 SemVer 规则^[34]。其中,SemVer 规则规定的版本格式为:主版本号、次版本号、修订号、版本号。递增规则如下。

1)主版本号(Major):当开发人员进行了不兼容的 API 修改,则需增加主版本号。

2)次版本号(Minor):当开发人员新增了向下兼容的功能,则需增加次版本号。

3)修订版本号(Patch):当开发人员修正了向下兼容的问题,则需增加修订版本号。

4)先行版本号及版本编译信息可以加到“Major. Minor. Patch”的后面,作为延伸。

2.5 相关工作

文献[2]立足于开源软件供应链,给出了开源软件供应链相较之前研究更为全面、更为准确的新定义,总结了开源软件

供应链的 4 个关键环节,并提出了开源软件供应链的模型。根据该模型,对供应链各个环节进行了风险分析并分别介绍了各个环节的风险识别技术。该文还总结分析了供应链的加固与防御方案,指出当下存在的挑战并前瞻性地提出未来发展方向。

文献[35]系统综述了 1990 年至今的与漏洞自动修复技术相关的研究,归纳了缓冲区溢出、整型溢出、安全验证错误型等特定类型漏洞以及通用类型漏洞的修复工作,并按照技术原理的不同将相关研究分类总结为四大类:静态分析的研究方法、动态分析的研究方法、历史修复的研究方法、混合式的研究方法。该文还讨论了当前漏洞修复领域面临的挑战并对该领域未来发展的方向提出了宝贵的建议。

文献[36]总结了 Android 应用分析与源代码漏洞检测的方法或工具,从静态分析、动态分析、混合分析 3 个方面分类并讨论了源代码和应用程序分析方法;总结了使用机器学习或常规方法的源代码漏洞检测方法,同时从静态、动态以及混合分析方面对这些方法进行了分析归纳;讨论了可能的漏洞预测方法与预防机制,以及一些用于分析源代码和检测漏洞的工具、存储库以及数据集。

这些研究或未聚焦于 OSS 组件漏洞检测与自动修复技术^[2],或侧重于总结应用程序源代码漏洞的自动修复方法,而未对在应用程序引入漏洞依赖项从而导致应用程序存在风险的情况下的修复方法进行总结^[35],或关注 Android 应用程序和源代码的分析与漏洞检测方法和工具^[36]。本文从 SCA 工作流程着手,总结并分类了 OSS 组件漏洞检测技术与自动化修复技术,选取并分析了 71 项研究,系统地介绍了相关背景知识、目前的研究方法与研究进展,并分析比较了部分 SCA 工具。

3 OSS 组件漏洞检测

现今在软件开发过程中,OSS 组件的使用无处不在^[37]。第三方库在当代软件开发中的广泛应用导致了大型相互依赖网络的创建,其中单个库的可持续性可以产生广泛的网络效应^[38]。因此,OSS 组件漏洞检测需要考察是否存在漏洞的依赖项,从而将风险引入软件项目。OSS 组件漏洞检测方法目前主要有 4 种:基于依赖元数据的检测、基于源代码的检测、基于二进制文件的检测和混合式的检测。这几种方法都具有各自的特点,总结归纳针对这些方法的研究具有重要意义,具体见 3.1—3.4 节。

此外,表 1 从 7 个方面(针对的编程语言、技术原理、分析方法、依赖深度、检测粒度、可达性以及局限性)总结了检测 OSS 组件漏洞的技术。在总结指标中引入可达性是因为部分存在漏洞的 OSS 组件在测试过程中被引入,但并未投入生产部署(程序执行时并未被使用)^[39],这种情况下,这些存在漏洞的 OSS 组件并不会导致依赖于它们的项目受到攻击。漏洞的可达性分析有的需要对应用程序的源代码进行分析^[40],有的通过模拟应用程序的实际执行判断漏洞是否可达^[41]。在 OSS 组件漏洞检测时考虑漏洞的可达性可以帮助相关人员了解漏洞的调用路径,也可以减少检测结果的误报。

表 1 OSS 组件漏洞检测方法总结概览
Table 1 Overview of OSS component vulnerability detection methods

文献	针对的 编程语言	技术 原理	分析方法		依赖深度	检测粒度	可达性	局限性
			静态分析	动态分析				
Decan 等 ^[42]	JavaScript	基于 依赖元 数据	✓	×	直接依赖	包级	×	只能检测直接依赖项的安全性
Liu 等 ^[43]	JavaScript		✓	✓	间接依赖	包级	×	忽略不会影响到软件项目的依 赖项,可能产生误报
Plate 等 ^[41]	Java		✓	✓	间接依赖	方法级	✓	在动态分析时,性能开销较大
Ponta 等 ^[40]	Java		✓	✓	间接依赖	方法级	✓	无法证明在实际情况中存在漏 洞的代码是否被执行
Horton 等 ^[44]	Python	基于 源代码	✓	✓	间接依赖	方法级	×	未知的特定依赖关系而导致的 推理失败的情况仍需解决
Chinthanet 等 ^[39]	JavaScript		✓	×	间接依赖	方法级	✓	由于元数据的限制,分析的结果 可能不准确
Woo 等 ^[45]	C/C++		✓	×	直接依赖	方法级	×	未考虑常见和琐碎函数等情况 会产生误报
Wu 等 ^[46]	C/C++		✓	×	直接依赖	方法级	×	目标软件没有正式的 BOM 文 件可能会影响 OSSFP 的精度
Jiang 等 ^[47]	C/C++	基于 二进制 文件	✓	×	直接依赖	方法级	×	未能考虑到所有非代码重用的 情况
Duan 等 ^[48]	Java,C/C++		✓	×	直接依赖	包级	×	未使用的 OSS 代码等情况可能 会导致结果不准确
Ohm 等 ^[49]	—		×	✓	直接依赖	包级	×	良性行为与是否是基于经验判断 的
Ding 等 ^[50]	汇编语言		✓	×	直接依赖	方法级	×	不适用于跨体系结构的语义 克隆
Pashchenko 等 ^[51]	Java	混合式 的检测	✓	×	间接依赖	方法级	✓	使用的漏洞数据库可能不会覆 盖所有已知的漏洞
Lauinger 等 ^[52]	JavaScript		✓	✓	直接依赖	包级	✓	可能存在动态检测签名无法检 测到库的情况
Tang 等 ^[53]	C/C++		✓	×	间接依赖	方法级	×	宏和编译设置可更改项目的依 赖项,静态分析不能识别

3.1 基于依赖元数据的检测

基于依赖元数据来识别可能存在的漏洞是一种通过分析目标软件的依赖关系和相关依赖元数据来确定可能存在漏洞的 OSS 组件的方法。在软件开发过程中,软件项目直接或间接依赖的开源安全组件数量众多,形成依赖嵌套^[2]。然而,手动检查每个依赖的开源软件组件是否存在漏洞是一项繁琐且耗时的任务。开源软件的维护通常是基于社区协作的过程,其中包含了开发人员、社区负责人、用户和组织的共同努力,并经常发布修复已知漏洞的更新版本,使得依赖元数据更新及时。因此,基于依赖元数据检测 OSS 组件漏洞能够发现所有已经公开的已知漏洞,帮助开发人员及时处理漏洞。利用依赖元数据来识别可能存在的漏洞成为一种更高效的方法。

Decan 等^[42]在 snyk.io 上收集了约 400 份安全报告,针对报告中涉及的漏洞软件,在 libraries.io^[54]上收集这些漏洞软件的版本列表,将这些漏洞软件引入作为依赖项的软件名称以及漏洞软件与直接依赖于漏洞软件之间的依赖约束。将从安全报告中分析获取的漏洞软件漏洞约束映射到相应的版本列表和依赖约束,最终确认了 133602 个直接依赖于漏洞软件 of 的软件。Kula 等^[55]通过 GitHub 获取使用 maven 构建的软件包,通过分析软件包中的 pom 文件获取直接依赖项,构建依赖关系,用于 Maven 生态系统的研究。

由于间接依赖的分析要求遵循依赖关系树的构造和特定的依赖管理系统的解析过程,因此,间接依赖的分析在技术层面上较为复杂。这些方法仅考虑直接依赖,没有考虑间接依赖关系^[42,55],并不适用于需要考虑间接依赖的场景。

项目中的间接依赖的占比约为直接依赖的 4 倍^[37],并且大多数漏洞会通过间接依赖影响到目标项目^[13]。由于间接依赖在开发过程中“隐形”了,因此开发人员可能难以发现间接依赖的软件包,这导致漏洞很容易由间接依赖被引入^[52]。因此,间接依赖应当被进一步研究。

Kikas 等^[12]研究 JavaScript,Ruby 和 Rust 生态系统的网络依赖关系,从存储库 npm、RubyGems 以及代码托管平台 GitHub 分别收集 JavaScript,Ruby 和 Rust 软件包,解析它们的依赖项文件,得到各自生态系统的依赖网络。考虑到解析 3 个包生态系统的间接依赖版本的时间花销与设备花销大,他们的研究在考虑间接依赖的版本时,没有使用解析版本而是使用特定版本。

Npm 包生态系统中,每个软件包会有一个 package.json 的配置文件,这个文件描述了项目的元数据和依赖关系,包含这个软件包的所有直接依赖项名称以及依赖约束。利用这个特性,Zimmermann 等^[56]收集了 npm 上的所有软件包,分析软件包中 package.json 文件中的信息,构建了依赖图。接着,从 npm 社区收集安全公告,将漏洞对应到依赖图上,从而检测出漏洞软件包与受漏洞软件包直接或间接影响的软件包。

Liu 等^[43]针对 npm 生态系统中忽略 npm 特定依赖关系解析规则导致错误依赖关系的情况,提出了一种基于知识图的依赖解析方法,确认漏洞传播路径,并在此基础上实现了针对 npm 包的基于依赖树的漏洞修复方法。该研究首先捕获了 npm 生态系统中所有包(超过 114 万个第三方库和1094万个版本)的依赖关系构建起依赖漏洞知识图(DVGraph);接着,基于 DVGraph 与 npm 的特有依赖解析规则来模拟依赖

项安装过程,通过这个精确的依赖解析算法(DTResolver)解析的依赖树与实际安装过程的依赖树的一致准确率被验证为 90%;最后,为不同的利益相关者提供了精确的补救措施(DTReme),该工具被证明能比 npm audit fix 处理更多的漏洞。

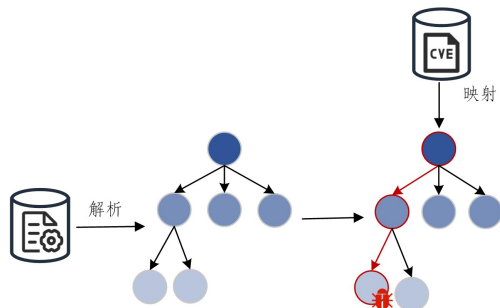


图7 基于依赖元数据的检测的工作模式

Fig.7 Working mode of detection based on dependency metadata

根据以上分析,可以总结出基于依赖元数据的 OSS 组件检测方法的一般工作模式,如图 7 所示。在该模式中,通常通过解析目标软件的配置文件等依赖元数据获取目标软件的依赖关系树,再利用 CVE 等漏洞数据库的元数据信息将已公开的漏洞映射到依赖关系树的节点,即目标软件的直接或间接依赖项,以此来检测目标软件是否受到漏洞的影响。

综上所述,基于依赖元数据的检测方法通常利用依赖元数据,如包管理器中的 BOM 文件、漏洞数据库中的漏洞信息等,生成软件项目的依赖关系图并将漏洞映射到依赖关系图中。这种方法无需进行代码分析等高成本过程,也易于自动化和集成,可确保开发人员在构建和部署项目时及时发现和解决安全问题,减少因安全问题而导致的延误或财务损失。然而,基于依赖元数据的 OSS 漏洞检测方法是基于以下假设:依赖元数据始终可用且准确,特别是与 OSS 组件(如名称、版本)和漏洞数据库(如受影响的软件名称及版本、漏洞详细数据)相关的依赖元数据。然而,这个假设是一种理想情况,Dong 等^[57]揭示了公共漏洞数据库的不一致性十分普遍,Bao 等^[58]手动验证漏洞的影响版本范围,发现 CVE 提供的漏洞信息很多是虚假的,Mu 等^[59]发现漏洞报告中缺失信息的情况普遍存在且漏洞可重复性低。这些发现使得基于依赖元数据的检测方法迎来新的挑战。在未来的研究工作中,需要考虑如何确保以及提高依赖元数据,尤其是漏洞数据库的信息质量,以提高识别漏洞的精确性,减少误报和漏报情况的发生。

3.2 基于源代码的检测

应用程序如果使用安全修复后,其所有依赖项的程序构造发生了改变,则应用程序存在漏洞。基于这一思想,Plate 等^[41]提出基于对安全修复引入的代码更改的分析评估依赖项可达性的方法,这种方法定义程序构造为描述源代码的结构元素,包括类型(如包、类、函数等)、语言(如 Java 等)和唯一标识符,将应用程序的所有依赖项的程序构造与安全修复后的程序构造进行比较,检测应用程序是否包含了漏洞代码。他们还跟踪应用程序与其所有依赖项的实际执行情况,检测应用程序是否包含且实际执行漏洞代码。

Ponta 等^[40]对上述方法^[41]进行了改进,给出以代码为中心、基于实际情况的方法评估依赖项的可达性。不同于文献^[41]对可达性只进行动态分析,该研究结合静态分析与动态分析对可达性进行评估。具体思想为:将漏洞的安全修复与应用程序及其所有依赖的程序构造与抽象语法树(AST)进行比较,静态地分析漏洞代码是否可访问以及其访问路径,在应用程序的实际执行过程中,动态分析漏洞代码是否被执行。该研究的贡献还在于针对受分析的应用程序,给出了更有可能最大限度减少更新工作和不兼容风险的建议。

Horton 等^[44]关注 python 项目的依赖解析问题,实现了 DockerizeMe 工具。该工具构建并解析源代码的 AST 以提取所有的导入资源(import 和 for import 节点)列表,并排除本地路径下的导入资源,接着通过查询知识库和包管理器将资源映射到包上。考虑到可到达的包,该方法还基于 DFS 维护了一个有序的包列表。实验证明了该工具能更好地解析依赖,也能克服没有直接名称匹配导致解析失败的困境。

Chinthanet 等^[39]为 Node.js 应用程序实现了基于代码的漏洞检测工具。他们在 SAP GitHub enterprise 上选取了 65 个应用程序,从 NVD 中选取了 100 个具有修复措施且最依赖 npm 包的漏洞并手动注释与漏洞修复对应的 commit 集合。接着通过解析 package-lock.json 文件获取应用程序的依赖关系树,遍历依赖关系树时使用 ANTLR-v4^[60]提取每个依赖的构造列表。为支持 JavaScript 代码的分析,他们扩展 Eclipse-Steady^[61]工具形成了 NAIST-SE/steady^[62],并利用此工具来确定应用程序是否包含了修复 commit 的代码更改,以此来确认应用程序是否被漏洞所影响。

C\C++ 没有统一的包管理器,也没有统一的 BOM 文件,现有的一些 SCA 工具如 OWASP Dependency Check^[63]对于 C\C++ 源代码无法检测。随着人们将目光移向 C/C++ 语言的第三方库的 OSS 组件分析,相关研究逐渐增多。Woo 等^[45]提出了 Centris——一种针对 C\C++ 语言的识别重复使用的 OSS 组件的方法,该方法对于修改后的 OSS 重用也同样有效。该方法提取每个 OSS 项目的所有函数,使用冗余消除方法排除掉每个项目不同版本之间的冗余,随之为这些函数生成唯一的签名,用于构建组件数据库。通过预设阈值判断目标软件与 OSS 间的代码相似性,从而识别重用的 OSS 组件。然而 Centris 未假设常见和琐碎函数,会影响该方法生成的签名。Wu 等^[46]考虑了这种情况,提出了针对 C\C++ 的 SCA 工具——OSSFP。该工具为针对 C\C++ 语言的 SCA 任务构建了全面的 TPL 数据库。首先为收集的 OSS 项目的每个函数生成特征,结合函数提交时间构建函数哈希索引,并删除同库不同版本间与不同库之间的重复函数,以为每个库构建库哈希索引,最后排除掉克隆功能、支持功能与常用功能,保留核心功能作为库的码纹(即唯一特征)。实验表明,该工具以高精度和高召回率检测 TPL,在 C\C++ 的 TPL 检测方面表现优秀。此外,Jiang 等^[47]在他们构建的数据集上评估 Centris 并得出结论:不准确的函数“birth time”与结果对阈值的依赖导致性能受到影响。由此,他们提出了 TPLite,利用源目录信息识别原始 TPL 提取正确的函数“birth time”,并实现了基于图的依赖项召回。以上工具和方法虽然是为了

解决 C/C++ 依赖关系的问题,但受这些研究启发,针对 C/C++ 的漏洞检测研究将进一步发展。结合以上研究,可以得知,基于源代码的 OSS 组件漏洞检测可以对源代码进行全面的分析,包括所有的功能、数据处理过程、执行逻辑等,也可以对源代码的执行情况进行观察,确定漏洞的可调用路径。在实际软件开发过程中,基于源代码的检测可以在集成到系统前发现漏洞,有利于降低系统的安全风险与修复成本。但对于庞大的软件项目,需要大量的人力资源及时间成本去完成检测工作,且大部分基于源代码的检测都是静态分析,静态分析难以回答“漏洞是否实际被利用”这一问题,动态分析可以解决这一困境,但是动态分析的性能开销较大。未来研究人员的潜在努力方向是将对源代码的静态分析与动态分析相结合,同时也需要在未来研究中致力于平衡性能与准确性的关系。另一方面, OSS 组件代码的广泛复用已逐渐揭示出安全方面的问题,因此构建庞大而全面的代码对比库也被认为是未来需要加以重视的一个研究方向。

3.3 基于二进制文件的检测

第 3.1 与 3.2 节提及的研究在进行 OSS 组件检测时,都需要软件项目的相关数据,可能是漏洞数据库,也可能是源代码,又或者是其他数据。有时,获取某些软件项目的数据存在困难,以上的方法均失效。还有一种方法——基于二进制文件的检测方法,它无需获取到软件项目的相关数据再提取 OSS 组件,可以直接对软件项目提取 OSS 组件。

针对移动应用, Duan 等^[48]实现了支持识别违规许可证以及存在漏洞的 OSS 组件的自动化工具,其名为 OSSPolice。该工具的核心思想是提取从 GitHub 等代码托管 web 服务处收集的 OSS 包的“birthmark”特征以构建一个支持高效查找的特征库,收集来自 Google Play Store 的二进制文件并生成它们的特征,在特征库中搜索得到匹配的 OSS 组件版本列表,最后对搜索结果进行进一步分析比对,识别出违规的许可证与存在的漏洞。

当恶意软件从软件项目被引入,系统通常会受到影响从而出现异常行为,如文件新增、注册表项改变等,这些行为可以通过数字取证证据捕获到。Ohm 等^[49]基于“恶意软件引起数字取证证据变化”这一假设,提出了一个提取第三方依赖的动态分析软件的框架——Buildwatch。该框架与一个开源沙箱——Cuckoo 集成,对软件的安装过程进行监控,并记录安装过程的文件(包括第三方依赖的安装文件)调用。通过检测与良性行为的差异判断可能存在的恶意行为,并且对已公开的恶意软件包进行分析,发现恶意软件倾向于新增包含恶意代码的文件并在新进程中执行。该框架的良性行为评估是基于经验总结的,可以将自动评估纳入未来工作。

Ding 等^[50]将语义纳入特征工程过程中,从汇编代码中学习语义关系,基于此实现了使用表示学习的语义相似的汇编代码克隆搜索工具——Asm2vec。该模型使用 IDA Pro2 反汇编器提取二进制文件的程序集函数、基本块和控制流图的列表,并将控制流图建模为多个对应潜在的执行跟踪的序列,通过学习汇编代码中词句的语义关系,将汇编函数表示为潜在语义的内部加权混合。该模型为单一汇编语言设计,不能适用于跨体系结构的语义克隆。

漏洞数据库中记录的漏洞代码片段通常是源码形式,当软件被发布到不同的操作系统中执行时是以二进制文件存在,因此检测二进制文件形式的软件在开发过程中是否通过代码克隆引入存在漏洞的 OSS 组件是一个极大挑战。对此, Wu 等^[64]提出了一种方法,名为 SourceSnipette2Binary。该方法将源代码片段和二进制代码转换为 LLVM IR 的方法,以实现统一的表达。接着,提取漏洞代码和目标二进制代码中关键路径的数据流,形成数据流链。对这些链进行归一化处理后,计算源代码与二进制文件之间的相似度,从而判断该二进制文件是否包含漏洞代码。

综合以上内容分析得出,基于二进制文件的检测独立性较高,它不需要依赖元数据,无需利用 BOM 文件等元数据进行 OSS 组件分析。二进制文件通常是机器代码,一般使用反汇编技术或监控安装过程得到依赖的第三方组件,适用性更强。此外,二进制文件占用的资源通常比源代码更少,不需要编译构建等过程,因此检测也更快。但二进制文件与源代码不同,它可能无法确认漏洞具体引入的位置导致误报,并且机器代码可读性差,难以理解。

3.4 混合式的检测

Hejderup 等^[65]发现解析出的依赖项并非所有都在实际执行过程中被调用,这意味着引入了漏洞依赖项的软件并不一定受到这个漏洞的影响。Latendresse 等^[66]发现超过 59% 的依赖项被引入软件开发过程但未被投入生产环境中使用,且静态扫描工具大多数警报针对的是未使用的依赖项,尽管它们几乎不可能对软件安全构成风险。针对以上研究所述情况, Pashchenko 等^[51]利用 maven 项目中的 pom 文件确认目标项目的依赖项清单,接着筛选排除未部署的依赖项,从而得到目标项目部署的所有依赖关系。他们指出 NVD 给出的漏洞影响范围太粗略,为避免此情况带来的影响,他们结合基于代码的检测,将 NVD、缺陷跟踪系统等来源获取的漏洞代码片段与收集目标项目及依赖项源代码进行代码层匹配,以此来检测漏洞是否真实存在。Pashchenko 等^[67]在之前的研究^[51]基础上,提出 Vuln4Real 方法来评估 FOSS 中易受攻击的依赖关系,通过过滤仅开发依赖项(仅在开发和测试阶段使用的依赖项,这类依赖项不会在生产环境中部署)识别项目间依赖(与项目中其他第三方库相关的依赖项)及确定滞后依赖项(本更新不活跃的依赖项,若是这类依赖项最新版本出现漏洞将很长一段时间不会收到修复程序),从而检测软件漏洞是否真实存在。

Lauinger 等^[52]采用库文件哈希值检测的方法进行研究。他们着眼于对 11 个已知存在漏洞的流行 JavaScript 库展开研究,建立了一个包含库文件哈希值、库的官方网站、Google 等维护的 JavaScript 内容分发网络(CDN)以及社区驱动的 CDN 下载链接的全面目录。研究团队下载了这些库的所有版本和变体(包括 debug 版本和 minified 版本),并根据不同版本和变体的库文件生成了唯一的哈希值,以便静态地检测使用这些库的软件是否受漏洞影响。由于 JavaScript 的特性,一个项目可能包含同一第三方库的不同版本,仅仅分析直接依赖关系并不能确保实际运行所使用的版本与直接依赖的版本一致。因此,研究还结合了动态分析方法,在实际运行过

程中检测使用这些库的软件是否使用的是受漏洞影响的版本。

根据研究调查,系统包管理器是广受开发人员欢迎的 C++ 依赖关系处理方法^[68]。因此,Tang 等^[53]设计 C/C++ 依赖关系检测器时考虑到了可能引入依赖的两种途径(包管理器 and 代码克隆)。针对包管理器引入依赖的途径,他们调研并总结了 21 个 C/C++ 包管理器并解析包管理器对应的 SBOM 文件获得依赖关系;针对代码克隆引入依赖的途径,他们集成了 Centris^[45]工具提取依赖关系,以此构建了 CCSscanner。该工具被证明能扫描大规模的 C/C++ 项目,但其准确性受到 Centris 工具的影响。虽然该工具是依赖关系检测器,但对 C/C++ 项目的漏洞检测工具的开发可以起到指导作用。

综上所述,混合式的检测中,研究人员考虑到各种方法的优点与局限性,将多种方法结合在一起进行检测分析,这是一项全新的尝试,在未来工作中可以尝试更多的多种方法组合用于检测 OSS 组件漏洞,或者针对不同语言特性找寻最适合的检测方法。

4 自动化修复

自动化修复应该包含安全补丁定位和兼容性检查两个中间过程,安全补丁定位确保及时找到适当的修复方案以解决漏洞,兼容性检查则能够验证修复措施不会引入新的问题或破坏系统原有的功能。通过结合这两个关键过程,可以有效提高自动化修复的准确性和可靠性,从而加强软件系统的安全性和稳定性。

与以“源代码中存在的缺陷或漏洞”为修复对象的研究^[35,69]不同,本文讨论的自动化修复的修复对象是 OSS 组件,它不仅是检测出的漏洞 OSS 组件,也可以是不兼容的 OSS 组件,还可以是过时的 OSS 组件。针对漏洞 OSS 组件,它的修复过程需要经过安全补丁定位和兼容性检查后,再将漏洞 OSS 组件更新到安全且兼容的版本;对于不兼容的 OSS 组件,则需要通过兼容性检查后,再更新到兼容的版本;而过时的 OSS 组件在其升级到新版本的过程中也需要考虑到兼容性,否则会导致软件构建或运行失败的问题。本文将在后文从安全补丁定位和兼容性检查两方面,介绍近年来自动化修复技术的研究进展。

4.1 安全补丁定位

漏洞的根本原因之一是缺乏安全最佳实践即补丁管理^[70],并且漏洞数据库中收录的补丁的覆盖率较低,缺失的补丁普遍存在^[71],能定位补丁的自动修复方法能够很好且很大程度上降低漏洞给整个应用程序带来的安全风险。安全补丁在本质上是由 OSS 开发人员在其它代码存储库中开发或者部署的代码提交,它没有更改软件预期设想的功能,无需测试就可以使用^[72]。在应用程序中检测出存在漏洞的 OSS 组件后,补丁定位的工作是需要确认依赖项的开发人员对修复该漏洞提供的补丁链接,以便及时、准确地修复漏洞,提高应用程序的安全性。现有的补丁定位方法主要可以分为两种:基于匹配和基于 commit 定位补丁(见 4.1.2 与 4.1.3 小节)。此外,也有研究人员利用机器学习等方法找寻补丁(见 4.1.4 小节)。近年来重要的安全补丁定位研究总结如表 2 所列。

表 2 安全补丁定位方法总结概览

Table 2 Overview of security patch location methods

文献	利用的依赖元数据	方法原理	局限性
Li 等 ^[19]	NVD,Git	从 NVD 中获取的外部引用链接识别补丁链接	只能找到少部分补丁
Kim 等 ^[73]	Git,CVE	在项目的 commits 历史中检索 CVE 标识符	
Tan 等 ^[74]	NVD,OSS 代码存储库	将 commit 与相应漏洞信息的相关性进行排序从而定位补丁	NVD 存在的信息缺失等问题可能导致补丁延迟
Xu 等 ^[71]	NVD,Debian,RedHat,Github	利用 NVD 等漏洞数据库的外部引用链接以及外部引用链接的内容中包含的链接构建深度为 5 的参考网络,从中识别补丁	对于没有 CVE 标识符的漏洞无法处理
Machiry 等 ^[72]	源代码文件	利用 PDG 比对补丁前和补丁后的函数,以此确认安全补丁	仅适用于 C 语言,应用于其他语言需要扩展
Wang 等 ^[75]	CVE,Github	构建安全补丁数据集并根据一组能区分补丁是否是安全补丁的特征识别安全补丁	构建安全补丁数据集时未能完全正确区分,对结果准确性有一定影响

4.1.1 补丁的管理与部署

探究补丁定位方法之前,有必要对“开源社区如何管理、部署补丁”这一问题进行介绍。一个 OSS 项目通常有多个分支,补丁可能会发布在多个分支,主要是考虑到以下因素。

- 1)版本特性差异。不同的分支对应着不同的版本,并且不同的版本可能存在差异,补丁需要针对不同版本的特性或漏洞来生成,这可以确保所有需要修复的版本都能有对应的补丁。
- 2)用户需求差异。由于某些原因,OSS 项目的用户可能不会一直使用最新版本^[13]。针对这些用户,开发人员需要维护多个稳定的分支来支持维护旧版本。稳定分支上的版本通常不再接受增加新功能,在稳定分支上的补丁也主要用于修复已知的漏洞或错误,确保旧版本的安全性,满足不同用户需求。

针对 OSS 社区中补丁的独特管理、部署模式,文献^[71]将漏洞与其补丁的映射关系定义为 3 种模式,具体如下。

- 1)One-to-one:一个漏洞只有一个补丁,一个漏洞只需要一个补丁就能修复。
- 2)One-to-some:一个漏洞有多个等效的补丁,任何一个补丁都可以修复漏洞。
- 3)One-to-many:一个漏洞有多个补丁,多个补丁一同作用修复补丁。

开源存储库的补丁从主分支散播到各个分支,有很长时间的延迟,并且一些安全修复缺少相关联的 CVE 条目^[72]。软件开发人员秘密修补漏洞而未创建 CVE 条目,一个合理的解释是:漏洞被公布导致被攻击者利用,从而损害软件的质量声誉^[75]。

文献[76]是第一个探究“多分支下的补丁管理实践”的研究,它将分支定义为3类:稳定分支、临时分支、镜像分支。他们深入研究这3种分支的补丁管理情况,发现了严峻的问题:并未打补丁的 CVE-Branch 对超过了80%,主要原因是该稳定分支不再维护或补丁管理故障;大多数分支的维护能力差,其中72.24%的稳定分支的漏洞补丁节点数量低于漏洞影响节点数量的50%;补丁移植时间长,其中23.19%漏洞的补丁移植时间超过30天。基于以上发现,该文献向OSS社区提出建议:应用自动补丁移植工具以及建立补丁部署状态监控机制。

4.1.2 基于匹配的方法

针对单个漏洞,CVE 以及 NVD 等其他漏洞数据库或安全公告平台通常会在“修复建议”字段给出该漏洞的补丁链接或其他补丁相关信息。基于匹配的方法则是根据这一思路,对于检测出的已存在漏洞,前往漏洞数据库或安全公告平台匹配漏洞,从而得到该漏洞的补丁链接或其他补丁相关信息。如文献[19]利用 NVD 从外部引用的参考链接中获取补丁 commit,并从对应的存储库中获取该项目的所有 commit,对比分析补丁 commit 与非补丁 commit,以研究补丁生命周期。

还有一种通过检索每个项目的 commit 消息中是否包含 CVE 标识符等关键字来判断是否是补丁 commit 的方法^[73],该方法也是基于匹配的方法。但由于开发人员很少在提交 commit 时打 CVE 标签,因此这种方法只能找到少部分的补丁。

近几年很少有基于匹配的方法用于找寻补丁,因为该方法匹配的结果只覆盖了少量的已经披露的补丁,不匹配时不会给出结果,但实际上不匹配的结果并不代表漏洞没有相应的补丁。

4.1.3 基于 commit 的方法

正如前面所提到的,安全补丁本质是代码提交。OSS 项目大多放在代码托管平台,如 GitHub。接下来,以 GitHub 为例子,解释为什么可以从 commit 定位到安全补丁。当开发人员发现开发软件存在漏洞代码并对漏洞代码进行一些操作(如增加、删除、修改)修复该漏洞后,需要将修复漏洞的代码更改上传到 GitHub。在上传代码更改时,GitHub 会生成一条 commit,它通常记录了修改的文件、修改的代码等相关信息。因此,安全补丁通常可以作为代码提交来找到。换言之,只要补丁存在,就可以通过探索所有 commit 来查找安全漏洞的补丁。图 8 给出了 jquery 的一条 commit 记录,它修复了 jquery-ui 中存在的 XSS 漏洞,是 CVE-2016-7103 的补丁。

Tan 等^[74]介绍了 PatchScout,其利用提出的漏洞-commit 相关排序方法来方便公开的 OSS 漏洞定位安全补丁。该方法采用模式匹配和命名实体识别(NER)从 NVD 和漏洞影响软件的所有 commit 中提取漏洞的信息元素,包括漏洞标识符(CVE-ID 与 Bug-ID)、漏洞位置(漏洞存在于哪个文件/哪个函数)、漏洞类型、漏洞详情,生成针对同一漏洞从 NVD 和 commit 中获取的信息元素之间的相关特征。最后,使用生成的相关特征训练一个基于 RankNet 的模型,以根据所有 commit 与特定漏洞的相关性进行排序。PatchScout 类似于搜索引擎将排序结果呈现给用户,用户需要根据 PatchScout 给出的排序结果手动检查每个结果是否是补丁。

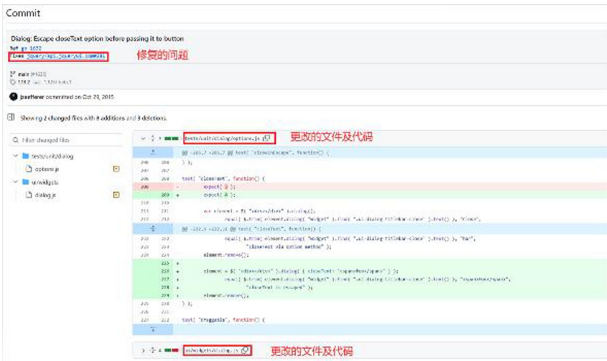


图 8 commit 形式的安全补丁

Fig. 8 Security patches in the form of commit

Xu 等^[71]提出了名为 tracer 的自动化方法,它可以从 NVD,Debian,Red Hat 和 GitHub 为 OSS 漏洞寻找补丁。它按照预先制定的规则提取 URL 引用,将 URL 分为3类:issueURL、补丁 URL、混合 URL(如博客 URL 等,这些链接的内容中可能指示了补丁的链接)。从 issueURL 和混合 URL 继续识别补丁 URL 直到建立起深度为5的参考网络,还对 GitHub 的 commit 历史进行搜索以增强参考网络。接着,以置信度(认为路径上包含 NVD 和 GitHub 节点的补丁节点具有高置信度)和连接性(认为到根节点路径最小的补丁节点连接性最好)作为准确、完整地选择补丁的标准。最后,通过同一存储库的跨分支搜索相关 commit 来扩展之前选择的补丁。该研究还从准确性、通用性、实用性证明了 Tracer 的价值与意义。

4.1.4 其他方法

除了前两小节所提到的基于匹配和基于 commit 的方法,也有研究人员考虑到这两种技术在不同场景下的局限性与缺点,提出了其他的方法用于定位补丁。

Machiry 等^[72]提出了基于代码的 SPIDER 技术,通过对比补丁前和补丁后的函数,识别哪些语句受到补丁的影响,紧接着识别并丢弃错误处理基本块,随之验证补丁后函数是否有增加输入空间以及输出是否等效以此确认安全补丁。由于该方法只考虑了语法层面的补丁适用性,找寻的补丁可能会造成语义冲突。

Wang 等^[75]使用机器学习来帮助识别安全补丁,他们首先根据 CVE 列表以及开源存储库的信息构建了安全补丁数据集与非安全补丁数据集,利用一组能区分安全补丁与非安全补丁的特征,使用投票算法将随机森林、贝叶斯网络、随机梯度下降、序列最小优化、自举汇聚法5个分类器的结果进行多数投票,从而进行安全补丁的识别。在他们的非安全补丁数据集中存在安全补丁,因此会对结果准确性产生一定影响。结合这次的经验,他们继续改进方法并构建了 PatchDB^[77]数据集。该数据集包括3种来源的安全补丁:NVD 收录的安全补丁、通过最近链接搜索算法扩充从 NVD 收集的安全补丁数据集的野生补丁,以及使用过采样技术生成的合成补丁。由于该方法中的数据依赖于 NVD,该结果受到 NVD 补丁的正确性的影响。

4.1.2—4.1.4 小节所述研究体现了安全补丁定位方法的多个方面,通过分析可以得出:基于匹配的方法定位补丁的

查全率极低,因为这种方法极度依赖漏洞数据库以及 commits 消息,对漏洞数据库收录的补丁完整性和 commits 消息中 CVE 标签的有无要求极高。反观现状,这显然是很难达到的。基于 commit 的方法通过从安全补丁的本质即代码提交中捕获补丁 commit,这种方法极大提高了补丁定位的精准性。还有一些研究人员另辟蹊径,利用机器学习等技术找寻补丁,这些方法提供了新的思路,但对准确性有所影响。

4.2 兼容性检查

以前的研究^[55]发现许多开发人员认为依赖项更新是额外的工作,因此依旧使用着过时的依赖项。依赖自动更新工具解决了这个问题,如 Dependabot。然而,一项研究^[78]发现虽然 Dependabot 确实可以依赖更新滞后的问题,但是它的兼容性分数太低,提供的依赖更新建议可能会导致不兼容问题。在 SCA 工具提供修复建议时,兼容性可能影响修复建议的用户采用率从而导致修复建议的可用性低,因此兼容性检查是有必要被考虑的。

兼容性检查指在修复漏洞的过程中,将受影响的 OSS 组件更新到新版本之前,检查新版本的 OSS 组件是否与目标软件中其他 OSS 组件兼容,以保持目标软件的正常功能。通过兼容性检查,可以解决漏洞修复过程中可能引发的兼容性问题,确保应用程序的稳定运行和与其他组件的互操作性。常见的不兼容问题主要体现在以下 3 个层面。

- 1)语法层面:如语法破坏,即在更改源代码或配置项文件时,引入了不符合语法结构的语句,从而导致应用程序编译错误或解析错误,甚至无法执行。
- 2)语义层面:如语义破坏,即在更改源代码或配置文件后,更改了某些文件的行为、功能或者逻辑(见图 9),导致应用程序的行为或输出与预期不一致,造成用户在版本升级、编译成功之后,仍然会遇到异常执行或崩溃的问题。

3)依赖项层面:如依赖冲突,即当应用程序依赖于同一依赖项的不同版本时(见图 10),可能会造成依赖项版本冲突的情况,这可能导致编译错误、运行时错误等情况。

```
124 125      task->thread.address = addr;
125 126      task->thread.error_code = ffs;
126 127      task->thread.trap_no = 14;
127 -      si.si_signo = SIGSEGV;
128 +      si.si_signo = sig;
128 129      si.si_errno = 0;
129 130      si.si_code = code;
130 131      si.si_addr = (void __user *)addr;
131 -      force_sig_info(SIGSEGV, &si, task);
132 +      force_sig_info(sig, &si, task);
132 133  }
```

图 9 语义更改示例
Fig. 9 Example of semantic changes

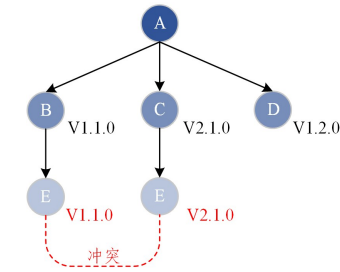


图 10 依赖冲突示例
Fig. 10 Example of dependency conflict

大部分安全研究人员专注于语义或依赖项层面,分别针对它们设计了相应的兼容性检查方法(见 4.2.1 与 4.2.2 小节)。也有安全研究人员同时将语义、依赖项等层面的问题考虑进研究中(见 4.2.3 小节)。由于大部分语法层面的不兼容问题在编译时能被检测到,编译器能按照严格的语法规范提出解决建议,因此本文没有讨论专注于语法层面的兼容性检查问题。关键的兼容性检查技术总结如表 3 所列。

表 3 兼容性检查方法总结概览
Table 3 Overview of compatibility check methods

方法名称	针对的编程语言	分析方法	支持修复的问题	重要贡献	局限性
Sensor ^[79]	Java	动态分析	语义冲突(SC)	一种用于检测 SC 问题的全自动技术	未能排除良性变化,存在误报情况
SEMBID ^[80]	Java	静态分析	语义破坏性更改(SemB)	第一个针对 SemVer 规则的静态 SemB 问题检测器	基于经验排除良性变化且未能定位实际使用的 API 导致结果的准确性受到一定影响
Riddle ^[81]	Java	静态分析、动态分析	依赖冲突(DC)	针对存在 DC 问题的项目,第一个自动生成测试并收集堆栈跟踪的方法	该方法的结果验证依赖于开发人员的反馈,且未对一些 DC 问题的表述形式进行分析
Watchman ^[82]	Python	静态分析	依赖冲突(DC)	高效检测并持续监控 PyPI 生态系统的依赖冲突问题	关注导致构建失败的 DC 问题,无法检测所有表征形式的 DC 问题
CORAL ^[83]	Java	静态分析、动态分析	语法破坏、语义破坏、依赖冲突	一个可兼顾兼容性与安全性对项目进行全局优化的第三方库修复工具	仅利用了静态调用图,故牺牲掉了部分准确性以获得更佳性能
DepOwl ^[84]	C/C++	静态分析	依赖冲突	一个在二进制级别检查应用程序与库兼容性的工具	需要依赖项的每个版本采用带有编译指令的源代码或者带有调试符号的二进制形式

4.2.1 专注于语义层面的检查

Wang 等^[79]提出了一种全自动的检测语义冲突问题的技术 Sensor,它使用 GUMTREE^[85]迭代检测在代码级别的细粒度差异上方法签名一致但行为不一致的冲突 API 对;并基于 EvoSuite^[86],使用遗传算法(GA)为每个引入已识别的异构冲突 API 的目标类迭代生成测试,以探究在实际执行中语

义冲突是否会被触发。实验表明,该方法的性能表现较好,并在工业场景中得到积极反馈。然而该方法不能排除良性变化的异构 API,仍然存在误报问题。

Zhang 等^[80]试图利用 SemVer 规则检查兼容升级(次版本和修订版本)上的语义破坏(SemB)问题,提出了第一个静态的 SemB 问题检测器——SEMBID。该方法首先生成 API

对的调用图,并将其中连续更改的方法分组为集群,接着建模输入与输出的关系,推导出数据依赖、控制依赖、异常 3 种依赖关系,并排除掉良性变化以避免假阳性。最后,构造了程序间语义图,利用 WL Graph Kernel 算法计算语义图的相似度,从而推断语义变化的程度以确定是否受 SemB 影响。实验结果表明,SEMBID 有 90.26% 的召回率和 81.29% 的准确率。但由于其良性变化排除是基于经验总结的,并且静态分析不能定位实际使用的 API 导致 API 范围不准确,因此结果的准确性受到一定影响。

4.2.2 专注于依赖项层面的检查

Wang 等^[87]将依赖冲突(DC)的表现形式分为库版本之间的冲突、库之间类的冲突,以及应用程序与库之间类的冲突 3 类,并总结了 DC 的 5 个主要修复形式。他们发现现有软件构建工具由于不能区分良性还是有害 DC 问题导致检测效果不佳,并实现了名为 Decca 的工具用于检测 Java 项目的 DC 问题,并评估 DC 问题的严重级别。他们推断出 DC 问题的特性集,并借此识别应用程序的依赖关系树、JAR 及类文件中重复的库或类是否存在 DC 问题,再根据这些问题的维护成本来判断其严重程度。但该方法的重点在于解决“shadowed features or classes”(引用被覆盖的变量或函数名)导致的 DC 问题。在此基础上,该团队进一步提出了 Riddle 工具^[81],将静态分析(Decca)与动态分析(条件突变、搜索策略、条件恢复)结合,为存在 DC 风险的项目自动生成测试并收集堆栈跟踪。

上下游项目中复杂的依赖关系导致的 DC 问题在 PyPI 生态系统中同样严重。Wang 等^[82]为自动监测 PyPI 中的 DC 问题提出了名为 Watchman 的技术。针对 PyPI 中的项目,Watchman 捕获并持续监控每个库的版本约束等元数据,构建并持续更新元数据存储库。随后,建模库之间的依赖关系,分析每个项目的完全依赖关系图来检测 DC 问题是否存在。该方法还可以预测两种基于经验总结的可以触发 DC 问题的类型。DC 问题的类型是基于人为经验得出的,可能会对该技术的分析结果产生一定影响。

依赖管理系统依赖于规范文件指定文件的版本范围,但 Jia 等^[84]发现规范文件存在依赖错误问题,这可能导致不兼容问题。他们针对 C/C++ 语言提出了兼容性检查工具 DepOwl,该工具在二进制级别检查目标项目与依赖项的不兼容问题,在依赖项的相邻版本收集不兼容更改,分析更改元素的用法,并检测每个依赖项的版本是否与目标项目不兼容,最终根据检测结果给出建议。然而 DepOwl 要求依赖项的每个版本采用带有编译指令的源代码或者带有调试符号的二进制形式,否则会产生漏报情况。

Caο 等^[88]实现了一个名为 PyCD 的工具,该工具准确提取 python 项目中的依赖信息,用于支持对依赖关系问题的研究。PyCD 的实现是从文献[44]中得到的启发且实验表明 PyCD 的精度和召回率表现优秀。该研究借助 PyCD 重点关注缺少依赖项、依赖项冗余、版本约束不一致 3 类依赖关系问题并探究它们的普遍程度、引入原因、演变过程,结果表明这 3 类依赖关系问题十分普遍,但在实际场景中管理它们却有许多困难。该研究对 python 中依赖关系可能存在的几类问

题做了详细的探索,可用于支撑依赖问题检查的后续研究。

此外,Cheng 等^[89]提出了 PcCRE 方法,生成领域知识图谱(KG)实现了对 python 兼容运行时环境的冲突感知推理。该方法的主要思想是通过获取要安装的第三方库或模块,查询他们构建维护的 KG 以确认最佳匹配的模块候选库,构建候选库的依赖关系图,以及进行依赖求解推断兼容的依赖版本。尽管 Cheng 等提出的方法针对的是“开源项目源代码可以在各公开平台直接访问,但其执行环境难以被正确恢复^[90]”的场景,但其可以对解决依赖冲突的依赖项自动修复工具提供指导性意见。

4.2.3 其他方法

Hejderup 等^[91]静态提取项目的直接和间接依赖并捕获项目与其依赖之间的调用路径,接着利用测试套件检测执行期间对所有依赖项的调用,以此测量测试套件对第三方库使用的覆盖率,结果表明用于自动依赖关系更新的测试套件并不可靠。为了减少依赖测试套件的自动依赖更新工具的误报更新问题,他们开发了一个名为 UPDATERA 的依赖项更改影响分析工具。该工具的核心思想是,将错误更新近似为项目执行流路径的变化,首先使用 AST 差分算法将当前依赖项与新版本依赖项的 AST 进行比较以识别潜在的行为更改,接着利用 WALA 算法构造调用图推断项目与依赖项的所有控制流路径,最后将潜在的行为更改与调用图相结合执行可达性分析。UPDATERA 被证明能检测到比测试套件多两倍多的错误更新,但其未能准确识别语义等价更改,并存在函数调用过度近似问题,仍然会导致误报。

Zhang 等^[83]提出的针对 Java 程序的依赖项兼容检查与修复方法,涵盖了语法破坏、语义破坏、依赖冲突等不兼容问题的检查。该方法首先分别利用 pom 文件与类文件建立依赖关系图与调用图,按照垂直分区与水平分区将依赖关系图分割成子图,并优化子图以找到可达性和未知漏洞最少的版本。为了避免次优解和死胡同,该方法实现了回溯机制;为了考虑全局最优方案,利用动态更新的调用图来实现实时的可达性分析。该方法被证明能够有效地平衡兼容性和安全性,并且在修复可达的漏洞、编译成功、单元测试成功方面均表现优秀。但由于为了追求更好的性能,只生成了静态调用图,牺牲掉了调用图的准确性。

综上可以得知,现有研究已经提出专门针对语义层面、依赖项层面以及涵盖多层面的兼容性检查方法,检查效果在各自的数据集上表现俱佳。然而,如何更全面且精确地识别语义、依赖项不兼容问题,以及如何更全面地考虑多种不兼容问题的自动化修复仍是亟待解决的难题。未来需要研究人员更多地针对实际项目中的不兼容性问题分别从语义、依赖项层面探索总结出兼容行为的模式与共性,以支撑解决多种兼容性问题的方法进一步发展,也需要持续钻研分析导致不兼容问题的原因,以警示开发人员规范开发流程。

5 SCA 工具

SCA 工具对 OSS 的漏洞管理实践起着支撑作用,它通过 OSS 组件漏洞检测与自动修复两个核心功能发现并修复软件项目中潜在的漏洞和风险。目前,有各种 SCA 工具助力学术

界的研究以及工业界的发展,如 Black Duck,steady,OWASP Dependency Check 等。这些工具各有所长,因此分析它们的特性有利于利益相关者在选取工具时结合用途使工具的效用发挥最大化。

5.1 分析维度

目前已有研究对 SCA 工具进行大规模的评估,Zhao 等^[6]提出了一个模型来评估 SCA 工具在 Java 项目中的依赖解析能力与有效性以及漏洞检测能力,并基于该模型在大规模的 Maven 项目上评估了 6 种先进的 SCA 工具。

与上述研究不同,本文未从 SCA 工具功能的精度(如依赖识别与漏洞检测能力)等进行大规模精准评估,而是站在用户的视角,从功能完整性与用户体验方面探讨现有 SCA 工具。本文将从以下 8 个方面对 SC 工具进行分析与总结。

- 1)工具性质:描述工具是开源性质还是商业性质。
- 2)针对的编程语言:描述工具支持分析什么编程语言的被测软件。
- 3)支持的扫描文件格式:描述工具支持的扫描文件格式是源代码文件还是二进制文件。在部分场景下,由于一些原因,应用程序的源代码无法获取或不能披露,支持二进制文件扫描就显得非常必要,因此将“支持的扫描文件格式”纳入分析准则。
- 4)分析方法:描述工具分析功能使用的分析方法是静态分析还是动态分析。
- 5)检测粒度:描述工具检测漏洞时的粒度,有依赖项级、方法级两种粒度。由于有研究发现依赖级别(包级)的分析会产生误报^[92],因此将“检测粒度”纳入对 SCA 工具分析的

- 准则中是有必要的。
- 6)可达性:描述工具在检查漏洞时是否考虑了可达性。在第 3 章已经描述可达性对漏洞检测的重要性,因此讨论 SCA 工具的“可达性”是有必要的。
- 7)可修复性:描述工具是否提供修复建议,修复建议可能包括漏洞检出后的补丁建议或依赖项更新建议。
- 8)兼容性:描述工具在提供补救措施时,是否考虑了兼容性。

5.2 SCA 工具选取

当前市场上存在大量 SCA 工具,本文为选择合适的 SCA 工具进行探讨,定义了两项选取原则:1)该工具或其所属机构被两篇及以上论文提及;2)该工具提供下载使用或者提供试用。根据这两项选取原则,本文从 6 篇文献^[7,37,40,93-95]中收集了 5 款 SCA 工具,其中包括 2 种开源工具(OWSAP Dependency Check^[63]和 Eclipse steady^[61]),以及 3 种商业工具(White Source(现已更名为 Mend.io^[96])、Black Duck^[97]和 Snyk Open Source^[98])。根据 2023 年 Forrester 发布的 SCA 报告^[99],将 2 种商业工具 Sonatype^[100]和 Veracode^[101]补充进本文的探讨范围,并且这两个工具与前面选择的 Mend.io,Black Duck 和 Snyk Open Source 同为该报告中业内最具影响力的 SCA 工具之一。此外,我们了解到 Scantist 已充分集成了学术界的前沿研究。为了解学术界先进成果在工业界的实现效果,故将 Scantist^[102]纳入本研究讨论范围。最终,本文基于以上 8 个方面的分析准则,对选取的 8 款 SCA 工具以用户身份进行了分析与比较,重点关注功能的完整性与用户体验感,分析结果如表 4 所列。

表 4 SCA 工具的总结
Table 4 Summary of SCA tools

工具名称	工具性质	针对的编程语言	分析方法	支持的扫描文件格式		检测粒度		可达性	可修复性	兼容性
				源代码	二进制	依赖项级	方法级			
Black Duck	商业	Java,C#,C/C++等	静态、动态	●	●	●	●	●	●	⊙
OWASP Dependency Check	开源	Java, .NET, Ruby, Node.js, Python 等	静态	●	○	●	—	—	●	○
Eclipse steady	开源	Java,python	静态、动态	●	—	●	●	●	●	⊙
Mend.io	商业	Java, .NET 等	静态	●	⊙	●	—	—	●	—
Veracode	商业	Java, Ruby, python 等	静态、动态	●	⊙	●	⊙	⊙	●	⊙
Sonatype	商业	Java, JavaScript, Python, C#, GO 等	静态、动态	●	●	●	⊙	⊙	●	●
Scantist	商业	Java, JavaScript, Python, Ruby, PHP 等	静态、动态	●	●	●	⊙	⊙	●	●
Snyk Open Source	商业	JavaScript, Java, Python, .NET 等	静态	●	⊙	●	●	⊙	●	⊙

注:—表示不支持,○表示基本不支持,⊙表示不完全支持,●表示完全支持。关于使用这些符号的解释,有些 SCA 工具虽然支持漏洞的可达性分析等功能,但站在用户角度,这些功能的实现效果并不理想或者说可用性不高,故用这些符号表示后 5 个准则的实现与否以及实现效果。

除以上 SCA 工具外,还有一些工具如 OpenVAS 等。尽管它们提供了 SCA 的部分功能且被众多开发人员等利益相关者使用,但它们并未提供完整的 SCA 功能。还有一类静态代码分析工具如 Klocwork,Testbed,Cppcheck,Coverity 等,它们能筛查源代码中的各种缺陷,对提高产品质量、安全性等方面的价值不容忽视。然而,本文主要关注能提供依赖解析、

OSS 组件漏洞检测与自动修复功能的 SCA 工具,因此在对 SCA 工具进行总结时,并未将以上两类工具考虑进分析范围。

工具选取对本文有效性的威胁:1)在工具选取阶段,由于本文首先在文献中收集 SCA 工具,可能会错过一些优秀的工具,导致它们没有被分析;2)本文从 Forrester 报告补充选择

工具,依赖于该报告的权威性。因此,本文结果不能保证涵盖业内所有优秀的 SCA 工具。

5.3 结论与发现

根据前期的调研并结合表 4 的结果发现,目前 SCA 工具大多都能支持基础功能且基础功能完成质量较优,如覆盖多种语言尤其是 Java、源代码文件扫描、静态分析、提供可用性较高的修复建议,但也存在一些问题。

1)针对 C/C++ 语言,支持检测目标项目中 OSS 组件的漏洞的工具相对较少且实际检测效果不理想。这一现象在很大程度上影响了对 C/C++ 语言的软件项目的安全性和兼容性等重要问题的精确识别和解决。正如 3.2 节所述,C/C++ 作为传统的系统级编程语言,没有其他编程语言如 Java、Python 那样的统一的包管理器,也缺乏统一的 BOM 文件,因此,针对 C/C++ 代码的依赖关系分析和漏洞检测变得更加困难。现有的工具无法准确快速地识别 C/ C++ 实际项目中的 OSS 组件^[46],从而导致现有关于 C/C++ 语言的 OSS 组件漏洞检测方法实现效果不理想。开发人员在处理 C/C++ 项目时,往往需要依赖于各种不同的工具和方法来进行代码审查和漏洞扫描,这增加了工作的复杂性和耗时。

2)大多 SCA 工具并未充分考虑漏洞的可达性问题,或者无法将漏洞可达性的结果有效传达给用户。这种情况可能是由于将可达性集成到 SCA 工具的功能模块会带来较大的扫描时间和其他资源消耗。有一些以快速便捷为特点的 SCA 工具可能没有将漏洞的可达性纳入它们的工作范围,一方面,这可能导致用户对潜在风险的准确评估受到限制,无法全面了解漏洞的严重性和潜在影响,无法有效地采取相应的修复措施;另一方面,可能会导致疏忽修复关键漏洞或者误将次要漏洞视为优先处理的关键问题,进而影响软件的稳定性和安全性。

3)提供修复建议时,兼容性很少被考虑。一些 SCA 工具声称支持兼容性,但它们仅关注依赖冲突问题,而未对更深层次的语义破坏等问题进行全面检测。这种状况可能导致开发人员忽视了潜在的兼容性隐患,从而增加了软件在不同环境下出现问题的风险。对于这一情况,一个合理的解释是:解决兼容性问题确实需要投入更多资源,一些 SCA 工具团队仍需要更多的时间去筹备、考虑更多的关于检测兼容性问题的实践方案,包括对软件依赖项的详尽分析、全面测试以及必要时进行代码重构等工作。在软件开发过程中,如果能够更早地考虑兼容性因素,并且采取相应的策略和措施来解决潜在的兼容性问题,将有助于降低后期修复的成本,提高软件的质量和稳定性。

6 现有挑战与未来展望

从现有研究来看,OSS 组件漏洞检测与自动修复技术已取得一系列突破性成果,但这两方面工作的研究进展仍然难以达到高效且精确地检测与修复 OSS 组件的要求。因此,本文总结了现存的挑战并对未来可能的发展方向做出了展望。

6.1 现有挑战

结合上述对 OSS 组件漏洞检测和自动修复方法的分析,总结目前面临的问题与挑战。

1)OSS 组件漏洞检测的准确性受软件复杂性限制。当代软件系统的复杂性不断增加,以满足不断演进的实际业务需求。与此同时,OSS 组件在现代软件开发中广泛应用,导致依赖关系网络的规模日益庞大且结构错综复杂。针对这种大规模复杂软件的情况,现有的 OSS 组件检测技术存在不可忽视的误报率和漏报率。在实际使用中,这会导致无法全面保障软件的安全性并且给用户带来额外的工作。

2)补丁信息缺失。OSS 组件的开发团队很少在发布安全补丁时打上相应漏洞的 CVE 标签甚至不打修复标签,这无疑给定位安全补丁增加了大量工作,从而大大影响了定位安全补丁的准确性。

3)漏洞数据库关键信息的质量不佳。一方面,部分漏洞的补丁未被收录或完全记录在漏洞数据库,这可能是因为在公开漏洞数据库之前已经进行内部修复,但相关信息未被公开披露。另一方面,漏洞影响版本存在夸大或低估问题,这无疑给 OSS 组件漏洞检测方法的精确率带来了巨大的挑战。

4)兼容性检查的误报。尽管不少现有研究或对语义层面对对依赖项层面的不兼容问题提出了检测及修复方法,但很少有研究对不兼容问题的模式进行实证研究以及全面的总结。对不兼容问题的探索工作不足,导致现有研究提出的修复方法存在疏漏,如在研究语义层面的兼容性检查方法时无法排除一些良性语义变化,从而影响兼容性检查方法的精确性。

6.2 未来展望

通过对 OSS 组件漏洞检测和自动修复方法现存挑战的分析,本文将未来可能发展方向总结如下。

1)支持细粒度漏洞可达性分析的 OSS 组件漏洞检测。通过漏洞可达性分析可知被引入的组件是否被访问,可以很大程度上缓解 OSS 组件漏洞检测的误报漏报情况。一方面,如今的软件开发过程中使用的编程语言多样化,因此在 OSS 组件漏洞检测过程中,集成针对各种编程语言的漏洞可达性分析具有重要意义,特别是作为系统级编程语言的 C/C++ 语言。另一方面,由于漏洞数据库一般只给出漏洞对应的组件或版本,需要进一步将漏洞对应到函数或行,以支持细粒度漏洞可达性分析。

2)大语言模型辅助定位补丁。目前存在部分漏洞的补丁未被收录或被完全收录进漏洞数据库,要识别这些掩藏在代码中的补丁,未来工作中可以利用大语言模型分析和建模自然语言形式的漏洞报告与代码形式的补丁之间的联系,以实现在庞大的代码库中识别补丁。进一步地,在识别漏洞的基础上,大语言模型通过学习已有的漏洞和补丁数据,可以生成符合语法规范和安全性要求的补丁代码。

3)漏洞引入位置定位。漏洞影响范围不准确的主要原因是漏洞数据库如 NVD 通常采用保守规则:“如果版本 X 存在漏洞,那么它之前的所有版本都存在漏洞^[103]”。漏洞引入位置定位的研究可以提升漏洞影响范围的精确度。然而现有的漏洞引入位置定位方法在 C/C++ 项目中效果不佳,且不能在大规模项目上进行验证。在今后针对 C/C++ 语言的漏洞引入位置的工作中,可以利用 CPG 解决 C/C++ 项目在不同版本之间的代码映射问题。在未来,构建大型可供验证实验

的数据集也是一个需要的研究方向。

4)不兼容问题的原因与特征的探索。目前小规模简单地探索不兼容问题模式已不能排除多种良性语义变化与多类型的依赖冲突问题。在今后的工作中,一方面,深入探析引起不兼容问题的成因并进行归纳总结,以启示开发人员基于严格的标准进行开发流程;另一方面,为给兼容性检查方法的后续发展奠定基础,基于大规模的实际项目探究不兼容行为的模式与共性。

结束语 软件成分分析对维护开源软件供应链安全起到实质性支撑作用,越来越多的工业团队和公司意识到开源软件组件安全的重要性,并将 SCA 工具纳入产品的生产和维护过程。为加深相关人员对 SCA 在安全漏洞方面实践的了解,本文以 SCA 工具的工作流程为切入点,综述了 SCA 的两个核心功能——OSS 组件漏洞检测与自动修复技术的重要研究进展,并对这些研究进行了科学分类。从用户角度出发,对市场上常用的 8 个 SCA 工具进行了总结和分析。最后,本文从 OSS 组件漏洞检测和自动修复两个方面总结了现有研究所面临的挑战,并对未来可能的发展方向进行了展望,旨在为推动 SCA 领域的进一步发展提供参考。

参 考 文 献

- [1] 2023 China Software Supply Chain Security Analysis Report [EB/OL]. https://www.qianxin.com/threat/reportdetail?report_id=297/.
- [2] JI S L, WANG Q Y, CHEN A Y, et al. Survey on Open-source Software Supply Chain Security[J]. Ruan Jian Xue Bao/Journal of Software, 2023, 34(3): 1330-1364.
- [3] Log4shell [EB/OL]. <https://issues.apache.org/jira/browse/LOG4J2-3293/>.
- [4] State of Open Source Security 2022[EB/OL]. <https:// snyk.io/reports/open-source-security-2022/>.
- [5] WERMKE D, KLEMMER J H, WÖHLER N, et al. Always Contribute Back: A Qualitative Study on Security Challenges of the Open Source Supply Chain[C]// 2023 IEEE Symposium on Security and Privacy(SP). 2023: 1545-1560.
- [6] ZHAO L, CHEN S, XU Z, et al. Software Composition Analysis for Vulnerability Detection: An Empirical Study on Java Projects [C]// Proceedings of the 31th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. Association for Computing Machinery, 2023.
- [7] IMTIAZ N, THORN S, WILLIAMS L. A comparative study of vulnerability reporting by software composition analysis tools [C]// Proceedings of the 15th ACM / IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM). Association for Computing Machinery, 2021.
- [8] Software composition analysis [EB/OL]. https://en.wikipedia.org/wiki/Software_composition_analysis.
- [9] KITCHENHAM B, CHARTERS S. Guidelines for performing Systematic Literature Reviews in Software Engineering [EB/OL]. https://www.researchgate.net/publication/302924724_Guidelines_for_performing_Systematic_Literature_Reviews_in_Software_Engineering.
- [10] OWASP Top Ten[EB/OL]. <https://owasp.org/www-project-dependency-check/>.
- [11] YE H, CHEN W, DOU W, et al. Knowledge-based environment dependency inference for python programs[C]// Proceedings of the 44th International Conference on Software Engineering. Association for Computing Machinery, 2022: 1245-1256.
- [12] KIKAS R, GOUSIOS G, DUMAS M, et al. Structure and evolution of package dependency networks[C]// Proceedings of the 14th International Conference on Mining Software Repositories. IEEE Press, 2017: 102-112.
- [13] MIR A M, KESHANI M, PROKSCH S. On the Effect of Transitivity and Granularity on Vulnerability Propagation in the Maven Ecosystem[C]// 2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER). 2023: 201-211.
- [14] COX R. Surviving software dependencies[J]. Communications of the ACM, 2019, 62(9): 36-43.
- [15] IMTIAZ N, KHANOM A, WILLIAMS L. Open or Sneaky? Fast or Slow? Light or Heavy?: Investigating Security Releases of Open Source Packages[J]. IEEE Transactions on Software Engineering, 2023, 49(4): 1540-1560.
- [16] CHINTHANET B, KULA R G, MCINTOSH S, et al. Lags in the release, adoption, and propagation of npm vulnerability fixes [J]. Empirical Software Engineering, 2021, 26(3): 47.
- [17] LI X, MORESCHINI S, ZHANG Z, et al. The anatomy of a vulnerability database: A systematic mapping study[J]. Journal of Systems and Software, 2023, 201: 111679.
- [18] ALQAHTANI S S. A study on the use of vulnerabilities databases in software engineering domain[J]. Computers & Security, 2022, 116: 102661.
- [19] LI F, PAXSON V. A Large-Scale Empirical Study of Security Patches[C]// Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security. Association for Computing Machinery, 2017: 2201-2215.
- [20] DUSING J, HERMANN B. Analyzing the Direct and Transitive Impact of Vulnerabilities onto Different Artifact Repositories [J]. Digital Threats, 2022, 3(4): 1-25.
- [21] ZEROUALI A, MENS T, DECAN A, et al. On the impact of security vulnerabilities in the npm and RubyGems dependency networks[J]. Empirical Software Engineering, 2022, 27(5): 107.
- [22] IANNONE E, GUADAGNI R, FERRUCCI F, et al. The Secret Life of Software Vulnerabilities: A Large-Scale Empirical Study [J]. IEEE Transactions on Software Engineering, 2023, 49(1): 44-63.
- [23] WANG Y, SUN P, PEI L, et al. Plumber: Boosting the Propagation of Vulnerability Fixes in the npm Ecosystem[J]. IEEE Transactions on Software Engineering, 2023, 49(5): 3155-3181.
- [24] Apache Project Security Information[EB/OL]. <https://security.apache.org/projects/>.
- [25] Microsoft[EB/OL]. <https://msrc.microsoft.com/update-guide/vulnerability/>.
- [26] Openwall[EB/OL]. <https://www.openwall.com/lists/oss-security/>.

- [27] LI Z, YU Y, WANG T, et al. To Follow or Not to Follow: Understanding Issue/Pull-Request Templates on GitHub[J]. IEEE Transactions on Software Engineering, 2023, 49(4): 2530-2544.
- [28] CVE[EB/OL]. <https://cve.mitre.org/>.
- [29] NVD[EB/OL]. <https://nvd.nist.gov/>.
- [30] CNVD[EB/OL]. <https://www.cnvd.org.cn/>.
- [31] CNNVD[EB/OL]. <https://www.cnnvd.org.cn/home/loophole>.
- [32] OSV DB[EB/OL]. <https://osv.dev/list>.
- [33] GitHub Security Advisory[DB/OL]. <https://github.com/github/advisory-database/>.
- [34] SemVer[EB/OL]. <https://semver.org/>.
- [35] XU T T, LIU K, XIA X. Survey on Automated Vulnerability Repair[J]. Ruan Jian Xue Bao/Journal of Software, 2024, 35(1): 136-158.
- [36] SENANAYAKE J, KALUTARAGE H, AL-KADRI M O, et al. Android Source Code Vulnerability Detection: A Systematic Literature Review[J]. ACM Computing Surveys, 2023, 55(9): 1-37.
- [37] DANN A, PLATE H, HERMANN B, et al. Identifying Challenges for OSS Vulnerability Scanners—A Study & Test Suite[J]. IEEE Transactions on Software Engineering, 2022, 48(9): 3613-3625.
- [38] WATTANAKRIENGKRAI S, WANG D, KULA R G, et al. Giving Back: Contributions Congruent to Library Dependency Changes in a Software Ecosystem[J]. IEEE Transactions on Software Engineering, 2023, 49(4): 2566-2579.
- [39] CHINTHANET B, PONTA S E, PLATE H, et al. Code-Based Vulnerability Detection in Node.js Applications: How far are we? [C]// 2020 35th IEEE/ACM International Conference on Automated Software Engineering(ASE). 2020: 1199-1203.
- [40] PONTA S E, PLATE H, SABETTA A. Beyond Metadata: Code-Centric and Usage-Based Analysis of Known Vulnerabilities in Open-Source Software[C]// 2018 IEEE International Conference on Software Maintenance and Evolution(ICSME). 2018: 449-460.
- [41] PLATE H, PONTA S E, SABETTA A. Impact assessment for vulnerabilities in open-source software libraries[C]// 2015 IEEE International Conference on Software Maintenance and Evolution(ICSME). 2015: 411-420.
- [42] DECAN A, MENS T, CONSTANTINOU E. On the Impact of Security Vulnerabilities in the npm Package Dependency Network[C]// 2018 IEEE/ACM 15th International Conference on Mining Software Repositories(MSR). 2018: 181-191.
- [43] LIU C, CHEN S, FAN L, et al. Demystifying the vulnerability propagation and its evolution via dependency trees in the NPM ecosystem[C]// Proceedings of the 44th International Conference on Software Engineering. Association for Computing Machinery, 2022: 672-684.
- [44] HORTON E, PARNIN C. DockerizeMe: Automatic Inference of Environment Dependencies for Python Code Snippets[C]// 2019 IEEE/ACM 41st International Conference on Software Engineering(ICSE). 2019: 328-338.
- [45] WOO S, PARK S, KIM S, et al. CENTRIS: A Precise and Scalable Approach for Identifying Modified Open-Source Software Reuse[C]// 2021 IEEE/ACM 43rd International Conference on Software Engineering(ICSE). 2021: 860-872.
- [46] WU J, XU Z, TANG W, et al. OSSFP: Precise and Scalable C/C++ Third-Party Library Detection using Fingerprinting Functions[C]// 2023 IEEE/ACM 45th International Conference on Software Engineering(ICSE). 2023: 270-282.
- [47] JIANG L, YUAN H, TANG Q, et al. Third-Party Library Dependency for Large-Scale SCA in the C/C++ Ecosystem: How Far Are We? [C]// Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis. Association for Computing Machinery, 2023: 1383-1395.
- [48] DUAN R, BIJLANI A, XU M, et al. Identifying Open-Source License Violation and 1-day Security Risk at Large Scale[C]// Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security. Association for Computing Machinery, 2017: 2169-2185.
- [49] OHM M, SYKOSCH A, MEIER M. Towards detection of software supply chain attacks by forensic artifacts[C]// Proceedings of the 15th International Conference on Availability, Reliability and Security. Association for Computing Machinery, 2020.
- [50] DING S H H, FUNG B C M, CHARLAND P. Asm2Vec: Boosting Static Representation Robustness for Binary Clone Search against Code Obfuscation and Compiler Optimization[C]// 2019 IEEE Symposium on Security and Privacy(SP). 2019: 472-489.
- [51] PASHCHENKO I, PLATE H, PONTA S E, et al. Vulnerable open source dependencies: counting those that matter[C]// Proceedings of the 12th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement. Association for Computing Machinery, 2018.
- [52] LAUNGER T, CHAABANE A, WILSON C. Thou Shalt Not Depend on Me: A look at JavaScript libraries in the wild[J]. Queue, 2018, 16(1): 62-82.
- [53] TANG W, XU Z, LIU C, et al. Towards Understanding Third-party Library Dependency in C/C++ Ecosystem[C]// Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering. Association for Computing Machinery, 2023.
- [54] libraries.io[DB/OL]. <https://libraries.io/>.
- [55] KULA R G, GERMAN D M, OUNI A, et al. Do developers update their library dependencies? [J]. Empirical Software Engineering, 2018, 23(1): 384-417.
- [56] ZIMMERMANN M, STAICU C A, TENNY C, et al. Small world with high risks: a study of security threats in the npm ecosystem[C]// Proceedings of the 28th USENIX Conference on Security Symposium. USENIX Association, 2019: 995-1010.
- [57] DONG Y, GUO W, CHEN Y, et al. Towards the detection of inconsistencies in public security vulnerability reports[C]// Proceedings of the 28th USENIX Conference on Security Symposium. USENIX Association, 2019: 869-885.
- [58] BAO L, XIA X, HASSAN A E, et al. V-SZZ: Automatic Identification of Version Ranges Affected by CVE Vulnerabilities[C]// 2022 IEEE/ACM 44th International Conference on Software Engineering(ICSE). 2022: 2352-2364.
- [59] MU D, CUEVAS A, YANG L, et al. Understanding the repro-

- ducibility of crowd-reported security vulnerabilities[C] // 27th USENIX Security Symposium. USENIX Association, 2018: 919-936.
- [60] ANTLR[DB/OL]. <https://www.antlr.org/>.
- [61] eclipse steady[DB/OL]. <https://github.com/eclipse/steady>.
- [62] NAIST-SE/steady[DB/OL]. <https://github.com/NAIST-SE/steady/>
- [63] OWASP Dependency Check[DB/OL]. <https://owasp.org/www-project-dependency-track>.
- [64] WU Q, HUANG H, TANG Y, et al. Source Snippet Binary: A Method for Searching Vulnerable Source Code Snippets in Binaries[C] // 2021 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW). 2021: 288-289.
- [65] HEJDERUP J, BELLER M, TRIANTAFYLLOU K, et al. Präzi: from package-based to call-based dependency networks [J]. Empirical Software Engineering, 2022, 27(5): 102.
- [66] LATENDRESSE J, MUJAHID S, COSTA D E, et al. Not All Dependencies are Equal: An Empirical Study on Production Dependencies in NPM[C] // Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering. Association for Computing Machinery, 2023.
- [67] PASHCHENKO I, PLATE H, PONTA S E, et al. Vuln4Real: A Methodology for Counting Actually Vulnerable Dependencies [J]. IEEE Transactions on Software Engineering, 2022, 48(5): 1592-1609.
- [68] MIRANDA A, PIMENTEL J. On the use of package managers by the C++ open-source community[C] // Proceedings of the 33rd Annual ACM Symposium on Applied Computing. Association for Computing Machinery, 2018: 1483-1491.
- [69] DONG Y W, ZHANG H B, LI Y J. A codes reinforcement method for embedded software security vulnerability[J]. Aerospace Control and Application, 2021, 47(2): 17-24.
- [70] FADLALLAH Y, SBEITI M, HAMMOUD M, et al. On the Cyber Security of Lebanon: A Large Scale Empirical Study of Critical Vulnerabilities[C] // 2020 8th International Symposium on Digital Forensics and Security (ISDFS). 2020: 1-6.
- [71] XU C, CHEN B, LU C, et al. Tracking patches for open source software vulnerabilities [C] // Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. Association for Computing Machinery, 2022: 860-871.
- [72] MACHIRY A, REDINI N, CAMELLINI E, et al. SPIDER: Enabling Fast Patch Propagation In Related Software Repositories [C] // 2020 IEEE Symposium on Security and Privacy (SP). 2020: 1562-1579.
- [73] KIM S, WOO S, LEE H, et al. VUDDY: A Scalable Approach for Vulnerable Code Clone Discovery[C] // 2017 IEEE Symposium on Security and Privacy (SP). 2017: 595-614.
- [74] TAN X, ZHANG Y, MI C, et al. Locating the Security Patches for Disclosed OSS Vulnerabilities with Vulnerability-Commit Correlation Ranking[C] // Proceedings of the 2021 ACM SIG-SAC Conference on Computer and Communications Security. Association for Computing Machinery, 2021: 3282-3299.
- [75] WANG X, SUN K, BATCHELLER A, et al. Detecting "0-Day" Vulnerability: An Empirical Study of Secret Security Patch in OSS[C] // 2019 49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN). 2019: 485-492.
- [76] TAN X, ZHANG Y, CAO J, et al. Understanding the Practice of Security Patch Management across Multiple Branches in OSS Projects[C] // Proceedings of the ACM Web Conference 2022. Association for Computing Machinery, 2022: 767-777.
- [77] WANG X, WANG S, FENG P, et al. PatchDB: A Large-Scale Security Patch Dataset[C] // 2021 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN). 2021: 149-160.
- [78] HE R, HE H, ZHANG Y, et al. Automating Dependency Updates in Practice: An Exploratory Study on GitHub Dependabot [J]. IEEE Transactions on Software Engineering, 2023, 49(8): 4004-4022.
- [79] WANG Y, WU R, WANG C, et al. Will Dependency Conflicts Affect My Program's Semantics? [J]. IEEE Transactions on Software Engineering, 2022, 48(7): 2295-2316.
- [80] ZHANG L, LIU C, XU Z, et al. Has My Release Disobeyed Semantic Versioning? Static Detection Based on Semantic Differencing[C] // Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering. Association for Computing Machinery, 2023.
- [81] WANG Y, WEN M, WU R, et al. Could I Have a Stack Trace to Examine the Dependency Conflict Issue? [C] // 2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE). 2019: 572-583.
- [82] WANG Y, WEN M, LIU Y, et al. Watchman: Monitoring Dependency Conflicts for Python Library Ecosystem[C] // 2020 IEEE/ACM 42nd International Conference on Software Engineering (ICSE). 2020: 125-135.
- [83] ZHANG L, LIU C, XU Z, et al. Compatible Remediation on Vulnerabilities from Third-Party Libraries for Java Projects [C] // 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE). 2023: 2540-2552.
- [84] JIA Z, LI S, YU T, et al. DepOwl: Detecting Dependency Bugs to Prevent Compatibility Failures[C] // 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE). 2021: 86-98.
- [85] FALLERI J R, MORANDAT F, BLANC X, et al. Fine-grained and accurate source code differencing[C] // Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering. Association for Computing Machinery, 2014: 313-324.
- [86] EvoSuite[DB/OL]. <https://www.evosuite.org/>.
- [87] WANG Y, WEN M, LIU Z, et al. Do the dependency conflicts in my project matter? [C] // Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. Association for Computing Machinery, 2018: 319-330.
- [88] CAO Y, CHEN L, MA W, et al. Towards Better Dependency Management: A First Look at Dependency Smells in Python Projects[J]. IEEE Transactions on Software Engineering, 2023,

49(4):1741-1765.

[89] CHENG W, ZHU X, HU W. Conflict-aware inference of python compatible runtime environments with domain knowledge graph [C]//Proceedings of the 44th International Conference on Software Engineering. Association for Computing Machinery, 2022: 451-461.

[90] HORTON E, PARNIN C. Gistable: Evaluating the Executability of Python Code Snippets on GitHub [C]// 2018 IEEE International Conference on Software Maintenance and Evolution (IC-SME). 2018: 217-227.

[91] HEJDERUP J, GOUSIOS G. Can we trust tests to automate dependency updates? A case study of Java Projects [J]. Journal of Systems and Software, 2022, 183: 111097.

[92] ZAPATA R E, KULA R G, CHINTHANET B, et al. Towards Smoother Library Migrations: A Look at Vulnerable Dependency Migrations at Function Level for npm JavaScript Packages [C]// 2018 IEEE International Conference on Software Maintenance and Evolution (ICSME). 2018: 559-563.

[93] ANWAR A, ABUSNAINA A, CHEN S, et al. Cleaning the NVD: Comprehensive Quality Assessment, Improvements, and Analyses [J]. IEEE Transactions on Dependable and Secure Computing, 2022, 19(6): 4255-4269.

[94] MARANDI M, BERTIA A, SILAS S. Implementing and Automating Security Scanning to a DevSecOps CI/CD Pipeline [C]// 2023 World Conference on Communication & Computing (WCONF). 2023: 1-6.

[95] PONTA S E, PLATE H, SABETTA A. Detection, assessment and mitigation of vulnerabilities in open source dependencies [J]. Empirical Software Engineering, 2020, 25(5): 3175-3215.

[96] mend [DB/OL]. <https://www.mend.io/sca/>.

[97] Black Duck [EB/OL]. <https://www.synopsys.com/zh-cn/software-integrity/security-testing/software-composition-analysis.html>.

ware-integrity/security-testing/software-composition-analysis.html.

[98] Snyk Open Source [EB/OL]. <https:// snyk.io/product/open-source-security-management/>.

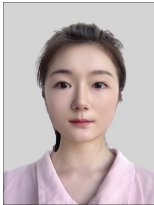
[99] Forrester [EB/OL]. https://www.forrester.com/report/the-forrester-wave-tm-software-composition-analysis-q2-2023/RES178483?ref_search=0_1711441921933.

[100] Sonatype [EB/OL]. <https://www.sonatype.com/products/open-source-security-dependency-management>.

[101] Veracode [EB/OL]. <https://www.veracode.com/products/software-composition-analysis>.

[102] Scantist [EB/OL]. <https://www.scantist.cn/product/sca>.

[103] NGUYEN V H, DASHEVSKYI S, MASSACCI F. An automatic method for assessing the versions affected by a vulnerability [J]. Empirical Software Engineering, 2016, 21(6): 2268-2297.



ZHANG Xuming, born in 2000, post-graduate. Her main research interests include software supply chain security and vulnerability analysis.



SHI Yaqing, born in 1981, Ph.D, professor, master's supervisor, is a member of CCF(No. 49805M). Her main research interests include intelligent software testing, temporal and spatial data processing.

(责任编辑:何杨)